PRESIDENCE DU CONSEIL

MINISTERE DE LA PLANNIFICATION DU DEVELOPPEMENT ET DE LA COOPERATION

Institut Africain d'Informatique
Etablissement Inter-Etats d'Enseignement Supérieur
Représentation du Togo (IAI-TOGO)
07 **BP** 12456 Lomé 07 **Tél.** : (+228) 22.20.47.00
E-mail : iaitogo@iai-togo.tg **Site Web**: www.iai-togo.tg

Support de cours

Conception et Implémentation des Structures de Données (Python)

Chargé de cours : M. **AKOMAGBE** Innocent

Dernière mise à jour : 22/04/2026

45 6e 20 64 65 70 61 73 73 61 6e 74 20 73 65 73 20 70 72 6f 70
72 65 73 20 6c 69 6d 69 74 65 73 20 6f 6e 20 72 65 61 6c 69 73
65 20 64 65 20 71 75 6f 69 20 6f 6e 20 65 73 74 20 72 65 65 6c
6c 65 6d 65 6e 74 20 63 61 70 61 62 6c 65

Année Académique 2025 - 2026

TABLE DES MATIERES

INTRODUCTION 4
PARTIE I: INTRODUCTION A PYTHON 5
Chapitre 1 : Prise en main de python **6**
I. Présentation de python 6
II. Carte d'identité de Python 6
III. Quelques caractéristiques de Python 6
IV. Installation des outils nécessaires 7
V. L'interpréteur de python 8
VI. Les unités lexicales 9
VII. Premier programme 10
Chapitre 2 : Les variables **13**
I. Affectation 13
II. Réaffectation 13
III. Affectations multiples 14
IV. Affectations parallèles 14
V. Règles de nommage d'une variable 14
VI. Les types de variables 15
VII. Écriture scientifique 15
VIII. Simplification de l'écriture de gros nombre 16
IX. Conversion 16
Chapitre 3 : Les entrées et sorties 17
I. Les entrées 17
II. Les sorties 17
Chapitre 4 : Les opérateurs 24
I. Quelques définitions 24
II. Les opérateurs de Python 24
Chapitre 5 : Les instructions conditionnelles 36
I. L'instruction if 36
II. L'instruction if … else 36
III. L'instruction if … elif … else 37
IV. L'instruction match 38
Chapitre 6 : Les instructions répétitives 41
I. La boucle while 41
II. La boucle for 42
III. Instructions de contrôle de boucle 43
Chapitre 7 : Les chaines de caractères 44
I. Création des chaines de caractères 44
II. Accès aux caractères individuels d'une chaîne 45
III. Suppression / Mise à jour d'une chaîne 46
IV. Formatage des chaînes 46

V. Parcours d'une chaine de caractères 47
VI. Accès au code ASCII 48
VII. Quelques fonctions de manipulation des chaines 48
Chapitre 8 : Les fonctions 53
I. Généralité 53
II. La déclaration d'une fonction 53
III. L'appel d'une fonction 53
IV. Notions de variables locales et globales 54
V. Un peu plus sur la définition des fonctions 55
VI. Arguments de longueur variable 56
VII. Fonctions anonymes 57
VIII. Fonctions récursives 58
IX. Chaînes de documentation 58
X. Les modules 59
Chapitre 9 : Les fichiers 61
I. Ouverture d'un fichier 61
II. Traitement sur un fichier 62
III. Fermeture d'un fichier 65
IV. L'instruction with 65
Chapitre 10 : Les exceptions 66
I. Généralités 66
II. La gestion des erreurs 66
III. Lever d'une exception 68
IV. Contrôles de saisies 68
V. Les assertions 68
PARTIE II: LES STRUCTURES DE DONNEES EN PYTHON 70
Chapitre 1 : Présentation et intérêt des structures de données 71
I. Quelques rôles essentiels des structures de données : 71
II. Implémentation des structures de données 71
Chapitre 2 : Les listes 73
I. Généralité 73
II. Déclaration d'une liste 73
III. Accès aux éléments d'une liste 73
IV. Opérations sur les listes 74
V. Liste de liste 80
VI. Les tableaux de type array 80
VII. Les listes en compréhension (List Comprehension) 81
Chapitre 3 : Les ensembles 83
I. Généralité 83
II. Création d'ensemble 83
III. Manipulation des éléments d'un set 83
IV. Opération sur les ensembles 85

V. Les frozensets 86
Chapitre 4 : Les tuples 87
I. Définition 87
II. Création d'un tuple 87
III. Opérations sur les tuples 87
IV. Application des tuples 90
Chapitre 5 : Les dictionaries 92
I. Définition 92
II. Création d'un dictionnaire 92
III. Opérations sur les dictionnaires 93
IV. Méthodes de dictionnaire 94
V. Liste de dictionnaires 96
VI. Les tris sur les dictionnaires 96
Chapitre 6 : Résumé des structures de données de type container 98
Chapitre 7 : Annexe 99
I. Quelques fonctions supplémentaires sur les chaines 99
II. Quelques méthodes disponibles avec l'objet file 101

INTRODUCTION

Le développement de système d'informations implique la manipulation de données devant être organisées de manière optimisée afin de faciliter les opérations comme la recherche, l'ajout de nouvelles informations ou la suppression. Les structures de données nous permettent d'atteindre cet objectif

L'étude des structures de données revêt une importance cruciale dans le domaine de l'informatique et du développement logiciel. Leur principal objectif est d'organiser et de stocker des données de manière efficace afin de faciliter leur manipulation, leur recherche et leur accès.

Ce cours enseignera dans un premier temps les fondamentaux du langage de programmation Python et ensuite l'implémentation des structures de données avec Python.

PARTIE I: INTRODUCTION A PYTHON

I. Présentation de python

Le langage python est un langage de programmation de haut niveau, interprété et facile à apprendre. Il dispose de structures de données de haut niveau et permet une approche simple mais efficace de la programmation orientée objet.

Grâce à sa syntaxe élégante et son typage dynamique, Python est un langage idéal pour l'écriture de scripts et le développement rapide d'application.



Figure 1: Logo de python

II. Carte d'identité de Python

Python est un langage de programmation interprété, multiparadigme et multiplateformes. Il favorise la programmation impérative structurée, fonctionnelle et orientée objet. Il a été créé par Guido van Rossum en 1989). Sa dernière version à l'écriture de ce document est le **`3.14.4`** publié le 7 avril 2026.

- **`Python 1.0`** est officiellement publié en janvier 1994 et ajoute des outils de programmation fonctionnelle tels que `lambda`, `map`, `filter` et `reduce`
- **`Python 2.0`** est publié en octobre 2000
- **`Python 3.0`** est publié en décembre 2008
- **`Python 3.10`** est publié en octobre 2021
- **`Python 3.14.0`** est publié en octobre 2025

III. Quelques caractéristiques de Python

Python est un langage de programmation polyvalent et populaire, apprécié pour sa simplicité, sa lisibilité et sa puissance.

- **Une lisibilité élevée :** La syntaxe simple, claire et intuitive de python favorise la lisibilité du code. Les blocs de code sont délimités par l'indentation, ce qui le code facile à comprendre pour les débutants.

- **Un langage interprété :** Python est un langage interprété, ce qui signifie qu'il n'a pas besoin d'être compilé avant d'être exécuté. Cela permet un développement plus rapide et un débogage plus facile.
- **Une bibliothèque standard riche :** Python dispose d'une vaste bibliothèque standard qui offre des modules et des packages pour effectuer une variété de tâches, telles que la manipulation de fichiers, le traitement des chaînes, la gestion des entrées/sorties, etc. Cela permet aux développeurs d'écrire des programmes complexes sans avoir à réinventer la roue.
- **Portabilité :** Python est un langage multiplateforme, c'est-à-dire qu'il peut être exécuté sur différentes plates-formes telles que Windows, MacOs et Linux sans nécessiter de modifications importantes du code source.
- **Dynamique et typage fort :** Python est un langage dynamiquement typé, ce qui signifie que les types des variables sont déterminés à l'exécution plutôt qu'à la compilation. Cependant, il possède un typage fort, ce qui signifie que les types des variables sont strictement contrôlés.
- **Grande communauté et écosystème :** Python bénéficie d'une vaste communauté de développeurs actifs qui contribuent à son écosystème en créant des bibliothèques, des frameworks et des outils qui étendent les capacités du langage.

IV. Installation des outils nécessaires

Avant de nous mettre à la programmation avec python, nous devons installer l'environnement avec les outils nécessaires.

1. Installation de l'environnement

Les fichiers d'installation de python sont disponibles sur le site web officiel de python : http://www.python.org/download.. Python étant un langage multiplateforme, il est possible de l'installer sur la plupart des systèmes d'exploitation.

2. Installation d'un éditeur de texte

Même s'il est possible d'utiliser l'interpréteur de python de façon interactive, il est pénible d'écrire tout un programme avec ce mode interactif. Dans le cadre de ce cours, nous utiliserons un éditeur de code pour l'édition de nos programmes.

Comme éditeur de code, nous disposons de VsCode, Eclipse, Pycharm, Notepad++, etc. Toutefois, nous choisirons VsCode pour sa simplicité, sa popularité et les fonctionnalités qu'il offre.

V. L'interpréteur de python

1. Le mode interactif de python

Python dispose d'un interpréteur en mode interactif qui nous permet d'écrire des instructions qui sont directement exécutées par l'interpréteur. Une fois l'instruction exécutée, les résultats sont affichés dans la console.

2. Lancement de l'interpréteur

Pour lancer l'interpréteur interactif de python, il suffit de saisir *python3* ou *python* dans ligne de commande sur MacOs et Linux, ou python sous Windows. Notons que Python dispose également d'un éditeur intégré nommé IDLE que nous pouvons également lancer.

Pour lancer IDLE, il suffit de rechercher et d'ouvrir l'application "IDLE Python" sous Windows.

Les caractères chevrons « >>> » également appelés *prompt* constituent le signal d'invite, ou

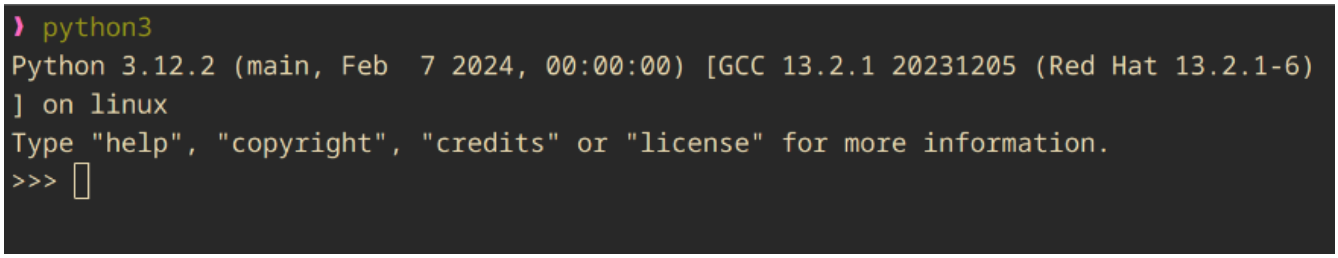


Figure 2: Exemple de lancement d'un interpréteur de Python

prompt principal, qui vous indique que Python est prêt à exécuter une commande.

En mode interactif, les commandes tapées sont immédiatement exécutées et le résultat est affiché. Mais le plus souvent les programmeurs préfèrent écrire le code de leur script Python contenant plusieurs instructions dans un fichier avec l'extension .py et sauvegardé sur un disque.

3. Python comme une calculatrice

L'interpréteur de Python peut être utilisé pour tester l'exécution de quelques lignes de code. Il peut être également utilisé comme une simple calculatrice. Les opérateurs +, -, * et / peuvent être utilisés pour effectuer des opérations arithmétiques. Les parenthèses (()) peuvent être utilisées pour grouper les opérations.

- **Quelques exemples de commandes**

```
>>>7+4
11

>>>7+3*4
19
#la hiérarchie des opérations mathématiques est bien respectée

>>>(7+3)*4
40
>>>20/33
0.6060606060606061

>>>4*2.5/3.3
3.0303030303030303
>>>5/2
2.5
# La division en Python retourne toujours comme résultat un réel
```

VI. Les unités lexicales

1. Les mots réservés

Python dispose de 33 mots réservés cités ci-dessous :

and	for	return	del	lambda
exec	print	continue	in	True
or	break	if	with	False
as	from	try	elif	except
finally	raise	def	is	not
pass	class	import	yield	
assert	global	while	else	

2. Les commentaires

Comme tous les langages de programmation modernes, Python autorise les commentaires dans un programme. Ils servent à la documentation du programme et sont destinés aux lecteurs humains. Ils n'ont aucune incidence sur le code et sont totalement ignorés lors de la compilation. Ils peuvent être placés à tout endroit d'un programme ou l'espace est autorisé et peuvent être sur une seule ligne comme sur plusieurs lignes.

Les commentaires en Python commencent avec un caractère croisillon, #, et s'étendent jusqu'à la fin de la ligne. Un commentaire peut apparaître au début d'une ligne ou à la suite d'un espace ou de code, mais pas à l'intérieur d'une chaîne de caractères littérale.

Exemple :

```
1. # une ligne de commentaire
2. print('Bonjour le monde') # Une autre ligne de commentaire
3. texte = "#  Ceci n'est pas un commentaire"
```

VII. Premier programme

Notre premier programme aura pour objectif d'afficher « **`Bonjour le monde`** » à l'écran.

- **Avec l'aide de l'interpréteur**

Dans votre interpréteur, tapez, le code suivant `print('Bonjour le monde')`.

```
1. >>>print('Bonjour le monde')
2. Bonjour le monde
```

On remarque que l'instruction est exécutée par l'interpréteur et la chaine de caractères « `Bonjour le monde` » est affiché.

- **Avec l'aide de l'éditeur de code VS Code**

Sur votre disque, dans un dossier Python, créer un fichier nomme `bonjour.py`. Ensuite, ouvrer ce fichier dans `VSCode` et réécrivez le code précèdent à l'intérieur du fichier.

Nous pouvons exécuter le script avec la commande suivante :

```
$ python bonjour.py
```

1. Structure d'un programme

a. Les instructions

Un programme Python est composé d'instructions. Une instruction est un ordre unitaire donné à un programme. Par exemple afficher « `Bonjour le monde` » est une instruction, de même qu'effectuer une opération.

La structure d'un programme Python est assez différente de celle des langages plus traditionnels comme le C.

En Python, les limites des instructions et des blocs sont définies par l'indentation[1].

- **Le point-virgule (`;`)** à la fin de chaque instruction est remplacé par le caractère de fin de ligne (symbolisé par `"\n"`). Ce qui simplifie la syntaxe, car traditionnellement les points-virgules étaient toujours suivis de retours à la ligne.
- **Les accolades (`{}`)** fréquemment utilisées pour définir les blocs sont remplacés par les niveaux d'indentation. En effet, il était déjà conseillé aux programmeurs de se servir aussi des sauts à la ligne et de l'indentation pour bien délimiter visuellement les blocs, donc c'est également un allègement du code.

En définitive, Python oblige à écrire du code lisible, et à prendre de bonnes habitudes conservables dans d'autres langages, ce qui en fait un langage idéal pour apprendre la programmation.

b. Les espaces sont ignorés.

À part ceux qui servent à l'indentation, en début de ligne, les espaces placés à l'intérieur des instructions sont ignorés, sauf s'ils font partie d'une chaîne de caractères.

L'interpréteur Python commence par analyser la première ligne :

- Si celle-ci contient une instruction, alors il l'exécute ;
- Si l'instruction n'est pas une instruction de contrôle, alors, il passe à la ligne suivante, l'analyse et l'exécute ;
- Si le programme Python arrive à la fin du fichier à exécuter, alors, il sort du programme et en arrête l'exécution.

2. Notion de bloc d'instructions et d'indentation

Un bloc d'instructions est une suite d'instructions qui est alignée sur la même tabulation. Les blocs d'instructions sont créés par les instructions de contrôles comme `if`, `while` et `for`, ainsi que par les instructions permettant de déclarer des fonctions.

Sous Python, toutes les instructions composées ont toujours la même structure : une ligne d'en-tête terminée par un double point, suivie d'une ou de plusieurs instructions indentées sous cette ligne d'en-tête.

[1] Indentation : Selon Wikipédia, l'indentation consiste en l'ajout de tabulations ou d'espaces dans un fichier, pour une meilleure lecture et compréhension du code. Le style d'indentation décrit les différentes manières que les programmeurs utilisent pour faire ressortir un bloc de code.

Nous disposons de deux façons pour indenter en Python : La première façon consiste à utiliser quatre espaces contrairement à la deuxième façon qui utilise un seul caractère tabulation. Cependant, il est interdit de mélanger les deux sous peine d'erreurs de type `IndentationError: unindent does not match any outer indentation level.`

La tabulation est notamment la convention utilisée pour la librairie standard de Python, mais les espaces ont l'avantage de fonctionner avec tous les éditeurs.

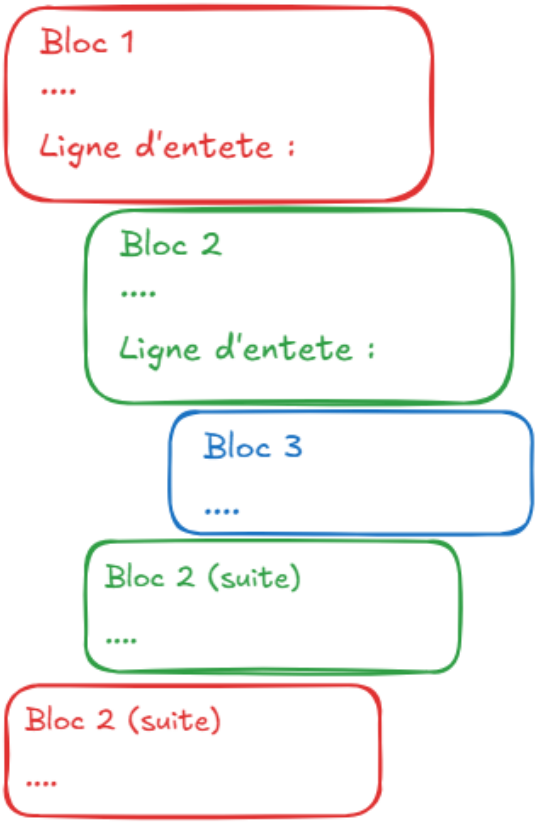


Figure 3: Schéma montrant les niveaux d'indentation en Python

Les résultats prévenants des calculs que nous effectuons dans l'interpréteur sont perdues après affichage. Pourtant, il pourrait être utile de conserver un résultat, afin de le réutiliser par la suite dans d'autres calculs.

Le concept des variables en Python est similaire à ceux des autres langages de programmation tels que C, Pascal, C++, Java. L'objectif est d'associer un nom à une valeur, dans le but de stocker cette valeur en mémoire RAM et pouvoir la retrouver (grâce à son nom).

Python ne dispose d'une instruction spéciale pour la déclaration des variables. La déclaration d'une variable se fait lors de son initialisation ou sa première affectation.

I. Affectation

En Python comme dans de nombreux autres langages, le symbole `'='` est l'opérateur d'affectation. Cette valeur restera accessible jusqu'à la fin de l'exécution du programme.

La valeur de la variable peut être un littéral, c'est-à-dire une constante, ou bien une expression (ex : une autre variable ou une instruction comme `''1+1''`). L'expression est évaluée avant d'être affectée à la variable.

Exemple :

```
jour = 'lundi'
```

NB : En Python, les constantes sont déclarées et affectées comme les variables. La convention PEP8 leur impose juste un nom en majuscules

II. Réaffectation

Si la variable affectée n'existe pas, l'ordinateur la crée, sinon il écrase sa valeur. En effet, il est permis de réaffecter une nouvelle valeur à une même variable, autant de fois qu'on le souhaite.

```
1. >>> jour = 'lundi'
2. >>> print(jour)
3. lundi
4. >>> jour = 'vendredi'
5. >>> print(jour)
6. vendredi
```

III. Affectations multiples

Sous Python, on peut assigner une valeur à plusieurs variables simultanément.

Exemple d'affectation multiple :

```
1. >>> x = y = 7
2. >>> x
3. 7
4. >>> y
5. 7
```

IV. Affectations parallèles

On peut aussi effectuer des affectations parallèles à l'aide d'un seul opérateur :

```
1. >>> a, b = 4, 8.33
2. >>> a
3. 4
4. >>> b
5. 8.33
```

V. Règles de nommage d'une variable

Sous Python, les noms de variables doivent en outre obéir à quelques règles simples :

- Un nom de variable est une séquence de lettres (a → z, A → Z) et de chiffres (0 → 9), qui doit toujours commencer par une lettre.
- Python est sensible à la casse (les caractères majuscules et minuscules sont distingués). Donc `"Joseph"`, `"joseph"`, et `"JOSEPH"` sont des variables différentes.
- Les 29 « mots réservés » du langage sont déjà pris (ex : `"print"`).
- Seules les lettres du code ASCII sont autorisées. Les lettres accentuées, les cédilles, les espaces, les caractères spéciaux tels que $, #, @, etc. sont interdits, à l'exception du caractère _ (souligné).

À partir de Python 3, les règles de nommage ont été assouplies, car il devient possible d'utiliser n'importe quelle lettre Unicode non-opérateur dans les noms. Par exemple, français = 1 est autorisé, mais pas f_en_€ = 6.55957.

Il faut également noter que les variables dont le nom commence par le caractère _ ont une signification particulière :

- Les noms commençant par un _ ne sont pas exportés lorsqu'ils se trouvent dans un module ;
- Les noms commençant par deux _ et finissant par deux _ sont réservés par le langage lui-même, notamment pour la programmation orientée objet.

VI. Les types de variables

Le type d'une variable correspond à la nature de celle-ci. Elle détermine les valeurs possibles qu'une variable peut posséder ainsi que les opérations qu'il est possible d'effectuer.

- Pour représenter les types numériques, nous avons :
  - **Le type entier (`int`)** permet de représenter n'importe quel nombre entier, peu importe sa taille.
  - **Le type flottant (`float`)** permet de représenter des nombres comportant une partie décimale, compris entre 10^{-308} et 10^{308}. La valeur spéciale `math.inf` représente l'infini.
  - **Le type complexe (`complex`)** permet de représenter des nombres complexes, où le nombre imaginaire se note `j`.
- Pour les chaînes de caractères, nous avons le type `str`

En dehors des types cités précédemment, Python dispose d'autres types comme les booléens, les sets, les dictionnaires, les listes, les tuples, etc.

VII. Écriture scientifique

On peut écrire des nombres très grands ou très petits avec des puissances de 10 en utilisant le symbole `e`.

```
1. >>>1e9
2. 1000000000.0
3. >>>3.14e-3
4. 0.00314
```

Exemple :

On appelle cela écriture ou notation scientifique. On pourra noter deux choses importantes :

- `1e6` ou `3.12e-3` n'implique pas l'utilisation du nombre exponentiel `e`, mais signifie 1×10^{6} ou 3.12×10^{-3} respectivement ;
- Même si on ne met que des entiers à gauche et à droite du symbole `e` (comme dans `1e6`), Python génère systématiquement un `float`.

VIII. Simplification de l'écriture de gros nombre

Il est parfois pénible de s'y retrouver quand on manipule de grand nombre ou un nombre avec plusieurs zéros. Afin de résoudre ce problème, Python autorise l'utilisation du caractère « souligné » (ou underscore) _ pour séparer des groupes de chiffres.

Exemple :

```
1. >>>salaire = 1_000_000
2. >>>salaire
3. 1000000
4. >>>nombre_avogadro = 6.022_140_76e23
5. >>>nombre_avogadro
6. 6.02214076e+23
```

IX. Conversion

Il est possible de convertir explicitement une donnée d'un type vers un autre, en utilisant des fonctions prédéfinies. Les fonctions `int`, `float`, `complex` et `str` permettent de convertir une donnée vers les types correspondants.

```
 1. >>> a = 14
 2. >>> print(type(a))
 3. <class 'int'>
 4. >>> b = 3.14
 5. >>> print(type(b))
 6. <class 'float'>
 7. >>> a = str(a)
 8. >>> print(a)
 9. 14
10. >>> b = int(b)
11. >>> print(b)
12. 3
```

Chapitre 3 : Les entrées et sorties

Les entrées et les sorties en Python sont fondamentales pour la communication entre l'utilisateur et le programme.

I. Les entrées

Les entrées (entrée) permettent à l'utilisateur de transmettre des données au programme, souvent à travers des fonctions telles que la fonction **`input`**, qui recueille les données sous forme d'une chaîne de caractères.

Pour récupérer la saisie de l'utilisateur, nous disposons de la fonction **`input`**. Elle renvoie une chaine de caractères lue depuis l'entrée standard. Nous pouvons ensuite utiliser les fonctions de conversions pour convertir cette chaine de caractères vers le type désiré.

```
>>>nom = input('Quel ton nom? ')
Quel ton nom? root
>>> print(nom)
root

>>> age = int(input('Quel est ton age? '))
Quel est ton age? 11
>>> print(type(age))
<class 'int'>
>>> print(age)
11
```

II. Les sorties

L'affichage à travers le périphérique de sortie standard se fait à partir de la fonction `print`. La fonction `print()` affiche l'argument qu'on lui passe entre parenthèses et un retour à ligne. Ce retour à ligne supplémentaire est ajouté par défaut. Si toutefois, on ne veut pas afficher ce retour à la ligne, on peut utiliser l'argument par « mot-clé » `end` :

```
1. >>> print('Bonjour le monde')
2. Bonjour le monde
3. >>> print('Bonjour le monde', end="")
4. Bonjour le monde>>>
5.
6. >>> print('Bonjour le monde', end="*\n")
7. Bonjour le monde*
```

La fonction `print()` peut également afficher le contenu d'une variable quel que soit son type. Par exemple, pour un entier :

```
1. >>> age = 11
2. >>> print(age)
3. 11
```

Il est également possible d'afficher le contenu de plusieurs variables (peu importe leur type) en les séparant par des virgules :

```
1. >>> nom ="abalo"
2. >>> print(nom, "a", age, "ans ")
3. abalo a 11 ans
```

1. Concaténation de chaines

Pour afficher deux chaînes de caractères l'une à côté de l'autre, sans espace, on peut soit les concaténer, soit utiliser l'argument par mot-clé sep avec une chaîne de caractères vide :

2. Écriture formatée

L'écriture formatée est un mécanisme permettant d'afficher des variables avec un certain format, par exemple justifiées à gauche ou à droite, ou encore avec un certain nombre de décimales pour les floats.

```
1. >>> print(nom + ' a ' + str(age) + ' ans ')
2. abalo a 11 ans
```

a. Les chaines de caractères formatées (f-strings)

`f-string` est le diminutif de `formatted` *string* `literals`. Elles nous permettent d'inclure la valeur d'expressions Python dans des chaines de caractères en les préfixant avec `f` ou `F` et écrire des expressions comme `{expression}`. Ce caractère `f` avant les guillemets va indiquer à Python qu'il s'agit d'une f-string permettant de mettre en place le mécanisme de l'écriture formatée, contrairement à une string normale.

Exemple :

```
1. >>> nom = 'abalo'
2. >>> age = 18
3. >>> print(f'{nom} a {age} ans')
4. abalo a 18 ans
```

b. Le spécificateur =

Le spécificateur = peut être utilisé pour développer une expression en « texte de l'expression, un signe égal, puis la représentation de l'expression évaluée » :

```
1. >>> nom = 'Abalo'
2. >>> prenom = 'Germain'
3. >>> age = 18
4. >>> print(f'{nom=} {prenom=} {age=} ans')
5. nom='Abalo' prenom='Germain' age=18 ans
```

c. Spécification de format

Les f-strings permettent de remplacer des variables au sein d'une chaîne de caractères. On peut par ailleurs spécifier le format de leur affichage. Le formatage avec `.xf` (x étant un entier positif) renvoie un résultat arrondi.

L'expression entre accolade peut être suivie d'un spécificateur de format. Cela permet un plus grand contrôle sur la façon dont la valeur est rendue. L'exemple suivant arrondit `pi` à trois décimales après la virgule :

```
1. >>> import math
2. >>> print(f'La valeur approximative de pi est {math.pi:.3f}')
3. La valeur approximative de pi est 3.142
```

NB :

- Le chiffre 3 après le point décimal indique la précision voulue, soit ici trois chiffres après la virgule.
- Un chiffre avant le point indique le nombre de caractères sur lequel écrire le nombre

d. Spécification de largeur

Il est possible de préciser sur combien de caractères, vous voulez qu'un résultat soit écrit et comment se fait l'alignement (à gauche, à droite ou centré).

Le caractère '>' spécifie un alignement à droite, '<' spécifie un alignement à gauche et '^' spécifie un alignement centré. Il est par ailleurs possible d'indiquer le caractère qui servira de remplissage lors des alignements (l'espace est le caractère par défaut).

Exemple :

Ce formatage est également possible sur des chaînes de caractères avec la lettre s (comme string) :

```
>>> print(10)
10
>>> print(1000)
1000

# Alignement à droite

>>> print(f'{10:>6d}')
    10
>>> print(f'{1000:>6d}')
 1000
>>>

# Alignement à gauche
>>> print(f'{10:<10d} a')
10         a
>>> print(f'{1000:<10d} a')
1000       a

# Alignement centre
>>> print(f'{1000:*^10d}')
***1000***
>>> print(f'{10:*^10d}')
****10****
```

```
>>> print(f'Prix {'TTC':>4s}')
Prix  TTC
>>> print(f'Prix {'HT':>4s}')
Prix   HT
```

e. Autres détails sur les f-strings

Si on veut afficher des accolades littérales avec les f-strings, il faut les doubler pour échapper au formatage :

```
>>> print ( f " Accolades littérales{{}} ou {{ ou }} et pour le formatage {10}")
 Accolades littérales {} ou { ou } et pour le formatage 10
```

Si on ne met pas de variable à formater entre les accolades dans une f-string, cela conduit à une erreur :

```
>>> print(f " accolades sans variable {}")
  File " < stdin >" , line 1
SyntaxError : f - string : empty expression not allowed
```

Il est important de bien comprendre qu'une f-string est indépendante de la fonction `print()`. Si on donne une f-string à la fonction `print()`, Python évalue d'abord la f-string et c'est la chaîne de caractères qui en résulte qui est affichée à l'écran.

f. Expressions dans les f-strings

Une fonctionnalité extrêmement puissante des f-strings est de supporter des expressions Python au sein des accolades. Ainsi, il est possible d'y mettre directement une opération ou encore un appel à une fonction :

```
>>> print(f"Le résultat de 5 * 5 vaut {5 * 5}")
Le résultat de 5 * 5 vaut 25
>>> print(f"Résultat d'une opération avec des floats : {(4.1 * 6.7)}")
Résultat d'une opération avec des floats : 27.47
>>> print(f"Le minimum est { min (1 , -2 , 4)}")
Le minimum est -2
>>> entier = 2
>>> print(f"Le type de { entier } est { type ( entier )}")
Le type de 2 est < class 'int ' >
```

3. Écriture scientifique

Pour les nombres très grands ou très petits, l'écriture formatée permet d'afficher un nombre en notation scientifique (sous forme de puissance de 10) avec la lettre e :

```
>>> print (f "{1 _000_000_000 : e }")
 1.000000 e +09
>>> print (f "{0.000 _000_001 : e }")
 1.000000 e -09
```

Il est également possible de définir le nombre de chiffres après la virgule. Dans l'exemple ci-dessous, on affiche un nombre avec aucun, 3 et 6 chiffres après la virgule :

```
1. >>> avogadro_ numbe r = 6.022_140_76e23
2. >>> print(f"{avogadro_number :.0 e}")
3. 6e+23
4. >>> print(f"{avogadro_number:.3 e}")
5. 6.022e+23
6. >>> print(f"{avogadro_number:.6 e}")
7. 6.022141e +23
```

4. Ancienne méthode de formatage des chaînes de caractères

a. L'opérateur %

L'opérateur % (modulo) peut également être utilisé pour le formatage des chaînes de caractères.

Pour **'string'** % values, les instances de % dans la chaine de caractères string sont remplacées par zéro ou plusieurs d'éléments de values. Cette opération est communément appelée interpolation de chaîne de caractères.

Exemple :

```
1. >>>age = 18
2. >>>print('Mon age est %s ans' % age)
3. Mon age est 18 ans
4.
5. >>>import math
6. print('The value of pi is approximately %5.3f.' % math.pi)
```

b. La méthode .format

Depuis la version **2.7** de Python, la méthode **.format()** (voir la rubrique suivante pour la définition d'une méthode) a apporté une nette amélioration pour mettre en place l'écriture formatée. Celle-ci fonctionne de la manière suivante :

```
1. >>> nom = 'Abalo'
2. >>> age = 18
3. >>> print("{} a {}  ans".format(nom, age))
4. Abalo a 18 ans
```

Pour conclure, ces deux anciennes façons de formater une chaîne de caractères avec l'opérateur % ou la méthode **.format()** vous sont présentées à titre d'information. La première avec

l'opérateur % est clairement déconseillée. La deuxième avec la méthode **`.format()`** est encore tout à fait valable. Python, conseille fortement aujourd'hui d'utiliser les `f-strings`.

Chapitre 4 : Les opérateurs

I. Quelques définitions

- **Un opérateur** est un symbole (ou un mot réservé) utilisé pour effectuer une opération entre des opérandes.
- **Un opérande** est une variable, un littéral ou bien une expression.
- **Une expression** est une suite valide d'opérateurs et d'opérandes.

Par exemple, dans l'expression : `x = y + 1`, il y a deux opérateurs (= et +) et trois opérandes (`x`, `y` et `1`).

Certains opérateurs peuvent avoir des comportements différents en fonction des types d'opérandes sur lesquels ils agissent : on parle alors de surcharge des opérateurs.

Exemples :

- L'opérateur + additionne des nombres, mais concatène des chaînes de caractères.
- L'opérateur * multiplie des nombres entre eux, mais duplique des chaînes de caractères.

II. Les opérateurs de Python

Python dispose de nombreux opérateurs qui peuvent être classés selon les catégories suivantes :

- Les opérateurs d'affectation ou d'assignation ;
- Les opérateurs arithmétiques ;
- Les opérateurs de chaînes ;
- Les opérateurs de comparaison ;
- Les opérateurs logiques ;
- Les opérateurs binaires ;
- Les opérateurs d'identité ;
- Les opérateurs d'appartenance.

1. Les opérateurs d'affectation

Les opérateurs d'affectation sont utilisés pour attribuer une valeur à une variable. La variable est l'opérande de gauche. La valeur est l'opérande de droite : il est évalué avant l'affectation. Nous disposons des affectations simples, parallèles et multiples.

En dehors de l'affectation à l'aide du symbole `'='`, Python dispose également de l'opérateur nommé opérateur d'affectation composé, ou augmenté qui combine une opération arithmétique ou binaire et une affectation de valeur à une variable, préalablement initialisée. Ces opérateurs nous permettent de réduire la taille de notre code à l'aide d'une écriture simplifiée.

Tableau 1: Résumé des opérateurs d'affectation augmentés de Python

Opérateur	Exemple	Équivalent à	Description
=	`x = 1`	`x = 1`	Affecte 1 à la variable x
+=	`x += 1`	`x = x + 1`	Ajoute 1 à la dernière valeur connue de x et affecte la nouvelle valeur (l'ancienne + 1) à x
-=	`x -= 1`	`x = x – 1`	Enlève 1 à la dernière valeur connue de x et affecte la nouvelle valeur à x
*=	`x *= 2`	`x = x * 2`	Multiplie par 2 la dernière valeur connue de x et affecte la nouvelle valeur à x
/=	`x /= 2`	`x = x / 2`	Divise par 2 la dernière valeur connue de x et affecte la nouvelle valeur à x
%=	`x %= 2`	`x = x % 2`	Calcule le reste de la division entière de x par 2 et affecte ce reste à x
//=	`x //= 2`	`x = x // 2`	Calcule le résultat entier de la division de x par 2 et affecte ce résultat à x
**=	`x **= 4`	`x = x ** 4`	Élève x à la puissance 4 et affecte la nouvelle valeur dans x
&=	`x &= 2`	`x = x & 2`	Effectue un et logique sur les bits des deux opérandes

|=	x |= 2	x = x | 2	Effectue un ou logique sur les bits des deux opérandes
^=	x ^= 2	x = x ^ 2	Effectue un ou exclusif logique sur les bits des deux opérandes
>>=	X >>= 2	x = x >> 2	Effectue un décalage des bits de x vers la droite de 2 pas
<<=	x <<= 2	x = x << 2	Effectue un décalage des bits de x vers la gauche de 2 pas

Depuis Python 3.8, l'opérateur morse *ou* `Walrus operator` noté : = réalise une affectation dans une expression.

Exemple :

```
1. x = 10
2. y = 15
3.
4. if (z := x + y) > 20:
5.   print(f'La somme de x et y est > 20 soit {z}')
6. else:
7.   print(f'La somme de x et y est < 20 soit {z}')
```

Ceci affiche : La somme de x et y est > 20 soit 25

2. Les opérateurs arithmétiques

Les opérateurs arithmétiques permettent d'effectuer les opérations mathématiques comme des additions, des soustractions, des multiplications, etc. entre différentes variables contenant des valeurs numériques.

Python dispose des opérateurs arithmétiques suivants :

Symbole	Opération	Types	Exemples
+	Addition	Entier, réel	2 + 5 == 7
-	Soustraction	Entier, réel	6 - 4 == 2

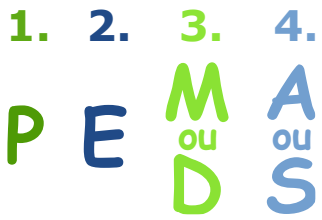
*	Multiplication	Entier, réel	`1.2 * 1 == 1.2` `2 * 5 == 10`
**	Puissance	Entier, réel	`12 ** 2 == 144`
/	Division	Réel	`6. / 4 == 1.5`
//	Division entière	Entier, réel	`6 // 4 == 1`
%	Modulo	Entier, réel	`6  %4 == 2`

NB : La division entre deux entiers produit un entier en Python 2, et un nombre à virgule flottante en Python 3.

a. Comportement de la division entière et du modulo selon le signe des opérandes

```
>>> 7 // 4
1
>>> -7 // 4
-2
>>> 7 // -4
-2
>>> -7 // -4
1
>>> 7 % 4
3
>>> -7 % 4
1
>>> 7 % -4
-1
>>> -7 % -4
-3
```

b. Ordre des opérations



En Python, l'ordre des opérations suit les mêmes principes que **PEMDAS**. C'est-à-dire que les expressions entre **Parenthèses** sont évaluées en premier, suivies des **Exposants**, puis de la **Multiplication** et de la **Division** (de gauche à droite), et enfin de **l'Addition** et de la **Soustraction** (de gauche à droite).

3. Les opérateurs de chaînes

Les opérateurs de chaînes vont nous permettre de manipuler des données de type `str` (chaînes de caractères) et par extension des variables stockant des données de ce type.
Python dispose de deux opérateurs de chaîne : l'opérateur de concaténation **+** et l'opérateur de répétition *****.

a. L'opérateur de concaténation +

L'opérateur de concaténation va nous permettre de mettre bout à bout deux chaînes de caractères afin d'en former une troisième, nouvelle.

Exemples :

```
1. >>> 'hakuna' + ' matata'
2. 'hakuna matata'
3. >>> age = 18
4. >>> nom = 'abalo'
5. >>> print(nom + ' a ' + str(age) + ' ans')
6. abalo a 18 ans
```

b. L'opérateur de répétition *

L'opérateur de répétition va nous permettre de répéter une chaîne un certain nombre de fois.

Exemples :

```
1. >>> 4 * 'b'
2. 'bbbb'
3. >>> 44 * 'b' + 2 * 'c'
4. 'bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbcc'
```

c. Les opérateurs de comparaisons

Les opérateurs de comparaison évaluent une expression et renvoient une valeur booléenne **`True`** ou **`False`**. Les opérateurs de comparaisons s'appliquent sur tous les types de base.

Opérateur	Description
<	Strictement inférieur
>	Strictement supérieur
<=	Inférieur ou égal
>=	Supérieur ou égal
==	Égal
!=	Différent

NB : Il est possible d'enchaîner les opérateurs : `X < Y < Z`, dans ce cas, c'est `Y` qui est pris en compte pour la comparaison avec `Z` et non pas l'évaluation de `(X < Y)` comme on pourrait s'y attendre dans d'autres langages.

Exemple :

```
1. # Exemple d'enchaînement des opérateurs de comparaison en Python
2. a = 5
3. b = 10
4. c = 20
5.
6. # Vérifie si 'a' est inférieur à 'b' et si 'b' est inférieur à 'c'
7. resultat = a < b < c
8. print(resultat)  # Cela affichera True, car 5 est inférieur à 10 et 10 est
inférieur à 2
9. True
```

4. Les opérateurs logiques

Ils sont utilisés pour combiner deux ou plusieurs conditions / contraintes ou pour compléter l'évaluation de la condition d'origine considérée.

Opérateur	Description	Fonctionnement
and	ET logique	Retourne `True` lorsque les deux conditions considérées sont remplies. Sinon, il retourne `False`.

`or`	OU logique	Renvoie la valeur `True` lorsque l'une (ou les deux) des conditions considérées est remplie. Sinon, il retourne `False`.
`not`	NON logique	Renvoie `True` si la condition considérée n'est pas remplie. Sinon, il retourne `False`.

NB :

- Ces opérateurs supportent n'importe quel type d'opérandes, car ils sont convertis implicitement en valeur booléenne pour l'évaluation :

Un entier est converti à `False` s'il vaut `0`, et à `True` sinon ;

Une chaîne de caractère, un tableau ou un dictionnaire est converti à `False` si elle est vide, et à `True` sinon. Il s'agit du même comportement que le précédent sur les entiers en utilisant le résultat de la fonction `len()` donnant la taille sous la forme d'un entier.

Exemples :

```
>>> bool('')
False
>>> bool('abc')
True
>>> bool(4)
True
>>> bool([])
False
>>> bool([4])
True
>>> bool('True')
True
>>> bool('False')
True
```

Pour les opérateurs **`or`** et **`and`** avec des opérandes non booléens, le comportement est donc le suivant

- **`X or Y`** : Retourne **`X`** si **`X`** équivaut à **`True`**, sinon retourne **`Y`**.

- **X and Y** : Retourne **X** si **X** équivaut à **False**, sinon retourne **Y**.

```
1. >>> 0 or 1
2. 1
3. >>> 0 and 1
4. 0
```

5. Les opérateurs binaires.

Les opérateurs binaires (ou bit à bit) appliquent des opérateurs logiques sur les bits de l'écriture binaire des opérandes.

Dans le tableau ci-dessous : Soit `x = 10` (`00001010` en binaire) et `y = 4` (`00000100` en binaire).

Opérateur	Signification	Exemple
&	**ET** logique	X & y = 0 (0000 0000)
|	**OU** logique	x | y = 14 (0000 1110)
~	**NON** logique	~x = -11 (1111 0101)
^	**XOR** logique	x ^ y = 14 (0000 1110)
>>	Décalage binaire à droite	X >> 2 = 2 (0000 0010)
<<	Décalage binaire à gauche	X << 2 = 40 (0010 1000)

6. Opérateurs spéciaux

Le langage Python offre des types d'opérateurs spéciaux, tels que l'opérateur d'identité ou l'opérateur d'appartenance.

a. Les opérateurs d'identité

Représentés par les opérateurs **is** et **is not**, ils permettent de vérifier si deux valeurs (ou variables) sont situées sur la même emplacement mémoire. Deux variables ayant les mêmes valeurs ne sont pas nécessairement identiques.

Opérateur	Signification	Exemple
is	True si les opérandes sont identiques (se référer au même objet)	x is True True

is not	**True** si les opérandes ne sont pas identiques (pas de référence pour le même objet)	x is not True

b. Les opérateurs d'appartenance

Représentés par les opérateurs `in` et `not in`, Ils sont utilisés pour tester si une valeur ou une variable est trouvée dans une séquence (chaîne, liste, tuple, set et dictionnaire).

Exemple :

```
1. >>> noms = ['lundi', 'mardi', 'samedi', 'dimanche']
2. >>> 'mardi' in noms
3. True
4. >>> 'jeudi' in noms
5. False
6. >>> 'dimanche' in noms
7. True
```

7. Priorités des opérateurs

Symbole	Type	Évaluation	Nom
{}	Groupe		Agencement de dictionnaire
()	Groupe		Agencement de n-uplet
[]	Groupe		Agencement de liste
.	Binaire	→	Attribut
()	Groupe		Argument de fonction
[]	Groupe		Partie (opérateur d'indiçage)
await	Unaire	←	Attente de résultat
**	Binaire	←	Puissance
~	Unaire	←	Inversion de bit
+	Unaire	←	Positif
-	Unaire	←	Négatif
*	Binaire	→	Multiplier

Symbole	Type	Évaluation	Nom
@	Binaire	→	Multiplication de matrices
/	Binaire	→	Diviser
//	Binaire	→	Résultat entier d'une division
%	Binaire	→	Modulo
+	Binaire	→	Addition
-	Binaire	→	Soustraction
<<	Binaire	→	Décalage à gauche
>>	Binaire	→	Décalage à droite
&	Binaire	→	et logique
^	Binaire	→	ou exclusif
|	Binaire	→	ou logique
in	Binaire	→	Test d'appartenance
not in	Binaire	→	Test de non appartenance
is	Binaire	→	Test d'égalité type
is not	Binaire	→	Test de non égalité de type
<	Binaire	→	Inférieur
>	Binaire	→	Supérieur
<=	Binaire	→	Inférieur ou égal
>=	Binaire	→	Supérieur ou égal
==	Binaire	→	Est égal
!=	Binaire	→	Est différent
not	Unaire	←	NON booléen
and	Binaire	→	ET booléen
or	Binaire	→	OU booléen
if ... else ...	Ternaire	→	Expression conditionnelle
lambda	Binaire	→	Expression lambda

Dans le tableau ci-dessus :

- Les opérateurs regroupés ayant les mêmes couleurs adjacentes ont la même priorité.
- Si deux opérateurs ont la même priorité, l'évaluation est effectuée de gauche à droite.
- Le sens d'évaluation indique l'ordre d'évaluation des opérations dans une expression :
  - → pour une évaluation de gauche à droite : `a OP1 b OP2 c == (a OP1 b) OP2 c;`
  - ← pour une évaluation de droite à gauche : `a OP1 b OP2 c == a OP1 (b OP2 c).`
- le type peut être :
  - « groupe » pour un couple de caractères encadrant une ou plusieurs expressions,
  - « binaire » pour un opérateur situé entre ses deux opérandes,
  - « ternaire » pour un opérateur utilisant trois opérandes,
  - « unaire » pour un opérateur précédant son unique opérande.

Exercice 1 :

Les nombres de Friedman sont des nombres qui peuvent s'exprimer avec tous leurs chiffres dans une expression mathématique.

Par exemple, 347 est un nombre de Friedman, car il peut s'écrire sous la forme $4 + 7^3$. De même pour 127 qui peut s'écrire sous la forme $2^7 - 1$.

1. $7 + 3^6$
2. $(3 + 4)^3$
3. $3^6 - 5$
4. $(1 + 2^8) \times 5$
5. $(2 + 1^8)^7$

Déterminez si les expressions suivantes correspondent à des nombres de Friedman. Pour cela, vous les écrirez en Python puis exécuterez le code correspondant.

Exercice 2 :

Essayez de prédire le résultat de chacune des instructions suivantes, puis vérifiez-le dans l'interpréteur Python :

```
1. (1+2)**3
2. "Da" * 4
3. "Da" + 3
4. ("Da"+"da") * 2
5. ("Da"*4) / 2
6. 5 / 2
7. 5 // 2
8. 5 % 2
```

Exercice 3 :

Essayez de prédire le résultat de chacune des instructions suivantes, puis vérifiez-le dans l'interpréteur Python :

```
1. str(4) * int("3")
2. int("3") + float("3.2")
3. str(3) * float("3.2")
4. str(3/4) * 2
```

Chapitre 5 : Les instructions conditionnelles

Le langage Python exécute normalement les instructions de la première à la dernière, sauf lorsqu'il rencontre une instruction conditionnelle comme l'instruction **if**. Une telle instruction va permettre au programme de suivre différents chemins suivant les circonstances.

Les instructions conditionnelles permettent de sélectionner le bloc d'instructions à exécuter selon la valeur d'une expression booléenne.

I. L'instruction if

Syntaxe

```
1. if <condition/expression>:
2.     <bloc d'instruction indentée 1>
3.     <bloc d'instruction indentée 2>
4. <bloc instruction indentée 3>
```

NB :

- `<condition/expression>` a pour valeur `True` ou `False`. Il s'agit de l'expression de la clause `if`
- Si expression a pour valeur `True`, les blocs d'instruction indentée `1` et `2` seront exécutés, sinon, ils sont ignorés.
- Dans tous les cas, le bloc d'instruction indentée `3` est exécuté ensuite.

Exemple :

```
1. nb = 8
2. if (nb % 2 == 0):
3.   print('nb est pair')
4.   print('parce que le reste de sa division par 2 est 0')
5. print('Fin')
```

II. L'instruction if … else

Syntaxe :

La syntaxe de l'instruction `if…else` est :

```
1. if <condition/expression>:
2.     <bloc d'instructions 1>
3. else :
4.     <bloc d'instructions 2>
5. <bloc instructions 3>
```

NB :

- **<condition/expression>** peut avoir la valeur **True** ou **False**
- Lorsque **<condition/expression>** a pour valeur **True**, le bloc **<bloc d'instructions 1>** est exécuté, et le bloc **<bloc d'instructions 2>** est ignoré.
- Lorsque expression a pour valeur **False**, le bloc **<bloc d'instructions 1>** est ignoré, et le bloc **<bloc d'instructions 2>** est exécuté.
- On exécute donc soit le bloc **<bloc d'instructions 1>**, soit le bloc **<bloc d'instructions 2>**, mais jamais les deux.
- Dans tous les cas, si le bloc **<bloc d'instructions 3>** existe, il est exécuté ensuite.

III. L'instruction if … elif … else

Dans le cas où l'on souhaite disposer de plus d'une alternative, l'instruction **if…elif… else** permet de conditionner l'exécution de plusieurs blocs d'instruction par la valeur de plusieurs expressions booléennes.

elif est équivalent à **else if**. On peut ajouter autant de clauses **elif** que nécessaires.

Syntaxe :

La syntaxe de l'instruction `if…elif…else` est :

```
1. if <expression 1>:
2.     <bloc d'instructions 1>
3. elif <expression 2> :
4.     <bloc d'instructions 2>
5. else:
6.     <bloc d'instructions 3>
7. <bloc d'instructions 4>
```

Remarque :

- Lorsque **<expression 1>** a pour valeur **True**, le bloc **<bloc d'instructions 1>** est exécuté, et les blocs **<bloc d'instructions 2>** et **<bloc d'instructions 3>** sont ignorés.

- Lorsque **<expression 1>** a pour valeur **False** le bloc **<bloc d'instructions 1>** est ignoré,
- Si **<expression 2>** a pour valeur **True**, le bloc **<bloc d'instructions 2>** est exécuté, et le bloc **<bloc d'instructions 3>** est ignoré ;
- Si **<expression 3>** a pour valeur **False**, le bloc **<bloc d'instructions 2>** est ignoré, et le bloc **<bloc d'instructions 3>** est exécuté.

On exécute donc soit le bloc **<bloc d'instructions 1>**, soit le bloc **<bloc d'instructions 2>**, soit le bloc **<bloc d'instructions 3>**, mais toujours un seul parmi les **3**.

Dans tous les cas, si le **<bloc d'instructions 4>** existe, il est exécuté ensuite.

Exemple :

```
>>> nb = 178
>>> if nb % 10 == 0:
...     print(nb, "est un multiple de 10")
... elif nb % 5 == 0:
...     print(nb, "est un multiple de 5, mais pas de 10")
... elif nb % 2 == 0:
...     print(nb, "est un multiple de 2, mais pas de 5, ni de 10 ")
... else :
...     print(nb, "n'est ni un multiple de 2, ni un multiple de 5 et ni de 10")
...
178 est un multiple de 2, mais pas de 5, ni de 10
```

IV. L'instruction match

L'instruction **match** confronte la valeur d'une expression à plusieurs filtres successifs donnés par les instructions case. Elle peut faire penser au **switch** que l'on trouve dans les langages C, Java, JavaScript et autres. Le premier filtre qui correspond est exécuté et elle permet aussi d'extraire dans des variables des composantes de la valeur, comme les éléments d'une séquence ou les attributs d'un objet. Elle permet de rendre les enchainements de **if/elif/else** simples à écrire et à lire. Cette instruction est apparue depuis la version Python **3.10**.

Syntaxe :

Sa syntaxe est la suivante :

```
match variable:
  case cas1:
    #Traitement à faire
  case cas2:
    #Traitement à faire
      #Les autres cas
  case _:
    #Traitement à faire
```

Exemple 1 :

```
print("""
Menu:
=====
Tapez:
  1 pour la somme
  2 pour le produit
  3 pour la moyenne
""")

choice = int(input('Quel est votre choix? '))

match choice:
    case 1:
        print('Vous avez choisi la somme.')
    case 2:
        print('Vous avez choisi le produit.')
    case 3:
        print('Vous avez choisi la moyenne.')
    case _:
        print('Vous avez effectuez un choix incorrect.')
```

Exemple 2 :

```
print("""
Menu:
=====
Tapez:
  1, s, S, + pour la somme
  2, p, P, + pour le produit
  3, m, M, / pour la moyenne
""")
choice = input('Quel est votre choix? ')

match choice:
    case 1 | 's' | 'S' | '+':
        print('Vous avez choisi la somme')
    case 2 | 'p' | 'P' | '*':
        print('Vous avez choisi le produit')
    case 3 | 'm' | 'M' | '/':
        print('Vous avez choisi la moyenne')
    case _:
        print('Vous avez fait un choix incorrect')
```

Application :

Compléter les différents cas de l'instruction match présent dans l'exemple 2 en suivant la logique suivante :

- On demande à l'utilisateur combien de nombres il souhaite saisir
- On réalise l'opération selon son choix

Chapitre 6 : Les instructions répétitives

Les structures répétitives, aussi connues sous le nom de boucles, permettent d'exécuter un bloc de code plusieurs fois. Python dispose de deux structures répétitives : La boucle `while` et la boucle `for`.

I. La boucle while

En python, la boucle `while` nous permet d'exécuter un bloc d'instructions plusieurs fois jusqu'à ce qu'une condition donnée soit remplie. Et lorsque la condition devient fausse, la ligne immédiatement après la boucle dans le programme est exécutée.

1. Syntaxe

```
1. while <condition/expression>:
2.     <instructions>
```

Toutes les instructions ayant le même niveau d'indentation (nombre d'espaces ou de tabulation), sont considérées comme faisant partie d'un seul bloc de code. Contrairement au langage C qui utilise les accolades, Python utilise l'indentation comme méthode de regroupement des instructions.

Rappels :

- Si la condition est fausse au départ, le corps de la boucle n'est jamais exécuté.
- Si la condition reste toujours vraie, alors le corps de la boucle est répété indéfiniment (tout au moins tant que Python lui-même continue à fonctionner), en risquant de saturer les processeurs (On parle d'une boucle infinie).

  Il faut donc veiller à ce que le corps de la boucle contienne au moins une instruction qui change la valeur d'une variable intervenant dans la condition évaluée par la boucle `while`, de manière que cette condition puisse devenir fausse et donc que la boucle se termine.

  Exemple :

```
 1. >>> i = 5
 2. >>> while i > 0:
 3. ...     print(i)
 4. ...     i = i - 1
 5. ...
 6. 5
 7. 4
 8. 3
 9. 2
10. 1
```

2. La clause else

Une clause `else` peut-être ajoutée à la boucle `while`. La clause `else` sera exécutée que lorsque la condition `while` devient `False`. Cependant, dans le cas où la boucle est arrêtée prématurément soit par l'instruction **`break`** ou un `return`, la clause `else` ne sera pas exécutée.

Exemple :

```
>>> i = 5
>>> while i > 0:
...     print(i)
...     i = i - 1
... else:
...     print('On est sorti.')
...
5
4
3
2
1
On est sorti.
```

```
>>> i = 5
>>> while i > 0:
...     print(i)
...     if i == 3:
...         break
...     i = i - 1
... else:
...     print('On est sorti')
...
5
4
3
```

On remarquera que l'instruction du else n'est pas exécutée dans ce cas à cause du l'instruction break.

II. La boucle for

La boucle for est utilisée pour les parcours séquentiels. Par exemple parcourir une liste, une chaîne, un tableau, etc.

1. Syntaxe :

```
for <iterateur> in <sequence>:
    <instructions>
```

NB : `<sequence>` peut être `Range`, `List`, `Tuple`, Dictionnaire, une chaine de caractères ou un fichier.

```
>>> for jour in ['lundi', 'mardi', 'mercredi', 'jeudi', 'vendredi', 'samedi', 'dimanche']:
...     print("Aujourd'hui c'est " + jour)
...
Aujourd'hui c'est lundi
Aujourd'hui c'est mardi
Aujourd'hui c'est mercredi
Aujourd'hui c'est jeudi
Aujourd'hui c'est vendredi
Aujourd'hui c'est samedi
Aujourd'hui c'est dimanche
```

2. La clause else

Une clause `else` peut être également ajoutée à la boucle `for`. Elle fonctionne de la même façon que celui de la boucle `while`.

3. Boucles imbriquées

Python permet d'utiliser une boucle dans une autre boucle (boucles imbriquées).

```
>>> for i in range(2):
...     for j in range(3):
...         print(f"{i=}, {j=}")
...
i=0, j=0
i=0, j=1
i=0, j=2
i=1, j=0
i=1, j=1
i=1, j=2
```

III. Instructions de contrôle de boucle

1. Les instructions break et continue

L'instruction `break` permet d'arrêter une boucle avant sa fin. L'instruction `continue` est similaire, mais au lieu d'interrompre la boucle, elle permet de passer à l'itération suivante.

```
>>> for i in range(5):
...     if i == 3:
...         break
...     print(i, end=" ")
...
0 1 2
```

```
>>> for i in range(5):
...     if i == 3:
...         continue
...     print(i, end=" ")
...
0 1 2 4
```

2. L'instruction pass

`pass` est une opération vide, quand elle est exécutée, rien ne se passe. Elle est utile comme bouche-trou lorsqu'une instruction est syntaxiquement requise, mais nous n'avons pas de code à exécuter. En langage C, cela équivaut à laisser les accolades d'une boucle for vides.

```
>>> for i in range(100):
...     pass
...
```

Chapitre 7 : Les chaines de caractères

En Python, les chaînes sont des tableaux d'octets représentant des caractères Unicode. Les chaînes de caractères (ou `str`) constituent un type de donnée composite, c'est-à dire, elle est composée d'un ensemble d'entités plus petites, en l'occurrence : les caractères.

A l'instar du langage C qui dispose d'un type caractère, Python n'en dispose pas. Un seul caractère est simplement une chaîne de caractères de longueur de 1. Les crochets peuvent être utilisés pour accéder aux éléments de la chaîne comme pour le cas des tableaux.

I. Création des chaines de caractères

Sous Python, une donnée de type string est une suite quelconque de caractères délimitée soit par des simples quotes (') soit par des doubles quotes ("), soit par des triples quotes (''') ou triples doubles quotes (""").

Exemple :

```
1. >>> texte = 'aeeeeeee'
2. >>> texte = "aeeeeeee"
3. >>> texte = '''aeeeeeee'''
4. >>> texte = '''
5. ...     aeeeeee
6. ...     aeeeeee
7. ...     zzzzzzz
8. ... '''
```

NB :

- Une chaîne entre quotes (') ne peut contenir aucun autre caractère quote ('), sinon une erreur se produirait, car le compilateur ne reconnaîtra pas où commencer et terminer la chaîne.

  Pour surmonter cette erreur, l'utilisation des doubles quotes (") est conseillée. On peut également l'échapper à l'aide du caractère backslash (\)
- Pour les chaînes contenant des doubles quotes ("), il est suggéré d'utiliser des triples quotes (''' ou """). Parallèlement à cela, les triples guillemets permettent également la création de chaînes multilignes.

II. Accès aux caractères individuels d'une chaîne

Python considère qu'une chaîne de caractères est un objet de la catégorie des séquences, lesquelles sont des *collections ordonnées d'éléments*. Cela signifie simplement que les caractères d'une chaîne sont toujours disposés dans un certain ordre. Par conséquent, chaque caractère de la chaîne peut être désigné par sa place dans la séquence, à l'aide d'un index.

1. Indexation

Le premier caractère (le plus à gauche) d'une chaîne de longueur n a pour indice `0` et le dernier l'indice `n-1`. L'indice est mis entre crochets.

Tableau 2 : Exemple d'indentation d'une chaine de caractères.

Chaine	B	O	N	J	O	U	R		L	E		M	O	N	D	E
(+) Indices	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
(-) Indices	-16	-15	-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

2. Sous-chaine

L'opérateur d'indiçage (`[]`) permet aussi de sélectionner des sous-chaines selon leurs indices. On appelle cette technique le slicing (« découpage en tranches »).

Dans la tranche :

- `[n]` indique le caractère à la position n
- `[n:m]`, est le caractère qui s'étend de l'indice **n** inclus à l'indice **m** exclus soit `m-1`.

Pour une chaîne de caractère **chaine**, les indices doivent être compris entre **`-len(chaine)`** et **`+len(chaine)-1`**. En dehors de cet intervalle, l'interpréteur retourne le message d'erreur suivant : **`IndexError: string index out of range.`**

Exemple :

```
>>> texte = 'Bonjour le monde'
>>> texte[0]
'B'
>>> texte[-1]
'e'
>>> texte[-2]
'd'
>>> texte[3:10]
'jour le'
>>> texte[2:-2]
'njour le mon'
>>> texte[0:-1]
'Bonjour le mond'
>>> texte[:-1]
'Bonjour le mond'
>>> texte[1:]
'onjour le monde'
>>> texte[0:len(texte)]
'Bonjour le monde'
```

III. Suppression / Mise à jour d'une chaîne

En python, les chaines de caractères étant immuables (non modifiable), la mise à jour ou la suppression de caractères d'une chaîne n'est pas autorisée, seules les nouvelles chaînes peuvent être réaffectées au même nom.

```
>>> texte = 'Bonjour le monde'
>>> texte[0] = 'b'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'str' object does not support item
assignment
>>> texte = 'b' + texte[1:]
>>> texte
'bonjour le monde'
```

IV. Formatage des chaînes

Les chaînes en Python peuvent être formatées à l'aide de la méthode `format()` ou aux `f-strings` étudiés précédemment.

V. Parcours d'une chaine de caractères

Il arrive très souvent que l'on doive traiter l'intégralité d'une chaîne caractère par caractère, du premier jusqu'au dernier, pour effectuer à partir de chacun d'eux une opération quelconque. Nous appellerons cette opération un parcours. Elle peut être réalisée grâce aux boucles `while` et `for`.

1. Grâce à la boucle while

```
>>> texte = 'aeeeeeee'
>>> i = 0
>>> while i < len(texte):
...     print(f'index = {i}, caractère = {texte[i]}')
...     i = i + 1
...
index = 0, caractère = a
index = 1, caractère = e
index = 2, caractère = e
index = 3, caractère = e
index = 4, caractère = e
index = 5, caractère = e
index = 6, caractère = e
index = 7, caractère = e
```

2. Grâce à la boucle for

a. Accès direct aux caractères

```
>>> texte = 'aeeeeeee'
>>> for c in texte:
...     print(f'caractère = {c}')
...
caractère = a
caractère = e
caractère = e
caractère = e
caractère = e
caractère = e
caractère = e
caractère = e
```

b. Accès aux indices et aux caractères

```
>>> texte = 'aeeeeeee'
>>> for idx, c in enumerate(texte):
...     print(f'index = {idx}, caractère = {c}')
...
index = 0, caractère = a
index = 1, caractère = e
index = 2, caractère = e
index = 3, caractère = e
index = 4, caractère = e
index = 5, caractère = e
index = 6, caractère = e
index = 7, caractère = e
```

VI. Accès au code ASCII

1. La fonction ord

La fonction `ord()` permet d'obtenir la valeur `ASCII` d'un caractère en Python. La fonction `ord()` prend un caractère unique en argument et renvoie la valeur `ASCII` correspondante.

Exemple :

```
>>> ord('a')
97
```

2. La fonction chr

La fonction `chr()` contrairement à la fonction `ord()`prend un entier en argument, qui doit être dans la plage valide des codes Unicode (de `0` à `1_114_111`, soit `0x10FFFF` en hexadécimal) et renvoie la chaîne de caractères correspondant au caractère Unicode dont le code est l'entier donné.

Exemple :

```
>>> chr(97)
'a'
```

VII. Quelques fonctions de manipulation des chaines

En Python, les chaînes de caractères sont des objets. Il est donc possible de faire de nombreux traitements à l'aide de méthodes appropriées. Nous pouvons obtenir la liste exhaustive de ces méthodes grâce aux fonctions `dir('')` ou `help(str)`.

1. La fonction len

Elle renvoie la longueur d'une chaîne de caractères passée en paramètre.

Syntaxe :

```
len(chaine)
```

Exemple :

```
1. >>> texte = 'Sandra'
2. >>> len(texte)
3. 6
```

2. La fonction count

Elle compte le nombre d'occurrences de la sous-chaîne dans la chaîne entière. Cette fonction prend 3 arguments, une sous-chaîne, une position de début (par défaut, 0) et une position de fin (par défaut, longueur de chaîne).

Syntaxe :

```
count("sous-chaîne", debut, fin)
```

Exemple :

```
1. >>> texte = 'aeeeeeee'
2. >>> texte.count('e')
3. 7
4. >>> texte.count('e', 3)
5. 5
6. >>> texte.count('e', 3, 4)
7. 1
8. >>> texte.count('e', 0, 3)
9. 2
```

3. La fonction isalpha

Elle renvoie **True** lorsque tous les caractères de la chaîne sont des alphabets, sinon renvoie **False**.

Exemple :

```
1. >>> texte = 'aeeeeeee'
2. >>> texte.isalpha()
3. True
```

4. La fonction isalnum

Elle renvoie **True** lorsque tous les caractères de la chaîne sont une combinaison de chiffres et / ou d'alphabets, sinon la valeur **False**.

```
>>> texte = 'aeeeeeee77'
>>> if texte.isalnum():
...     print('Cette chaine contient des caractères et des chiffres')
... else:
...     print('Cette chaine ne contient pas que des caractères et des chiffres')
...
Cette chaine contient des caractères et des chiffres
>>> texte = 'aeeeeeee7 7!'
>>> if texte.isalnum():
...     print('Cette chaine contient des caractères et des chiffres')
... else:
...     print('Cette chaine ne contient pas que des caractères et des chiffres')
...
Cette chaine ne contient pas que des caractères et des chiffres
```

5. La fonction isspace

Elle renvoie la valeur **True** lorsque tous les caractères de la chaîne sont des espaces, sinon la valeur **False**.

```
>>> texte = ' '
>>> texte.isspace()
True
>>> texte = ' -'
>>> texte.isspace()
False
```

6. La fonction join

Elle est utilisée pour joindre une séquence de chaînes mentionnées dans ses arguments avec la chaîne.

```
>>> mots = ["Tellement", "de", "choses", "qu'on", "promet"]
>>> ligne = ' '.join(mots)
>>> print(ligne)
Tellement de choses qu'on promet
```

7. La fonction split

La méthode `split()` en Python est utilisée pour diviser une chaîne de caractères en une liste de sous-chaînes. Par défaut, le séparateur est l'espace blanc (c'est-à-dire les espaces, les tabulations et les sauts de ligne).

```
1. >>> texte = "Le ciel prétend qu'il te connaît"
2. >>> print(texte.split())
3. ['Le', 'ciel', 'prétend', "qu'il", 'te', 'connaît']
4. >>> texte = "Jeanne|Juliette|Gloria|Gracia"
5. >>> noms = texte.split('|')
6. >>> print(noms)
7. ['Jeanne', 'Juliette', 'Gloria', 'Gracia']
8. >>> len(noms)
9. 4
```

8. La fonction startswith

Elle permet de retourner **`True`** si la fonction commence par la sous-chaine mentionnée (préfixe) sinon **`False`**.

La méthode `startswith('sous-chaine', debut, fin)` prend un argument obligatoire qui est la valeur à vérifier.

Optionnellement, nous pouvons fournir deux autres arguments :

- **`start`** : Un entier spécifiant la position à laquelle la recherche doit commencer (la valeur par défaut est **`0`**).
- **`end`** : Un entier indiquant la position à laquelle la recherche doit se terminer (la valeur par défaut est la fin de la chaîne).

```
>>> s = 'IAI-2024-02-01-10287'
>>> s.startswith('IAI')
True
>>> s.startswith('IAI-2024')
True
>>> s.startswith('ALGO')
False
>>> s.startswith('2024', 4)
True
>>> s.startswith('2025', 4)
False
```

Nous disposons également de la méthode **`endswith`** qui fonctionne exactement que la méthode **`startswith`** sauf qu'elle retourne **`True`** si la fonction se termine par la sous-chaine mentionnée (suffixe) sinon **`False`**.

Chapitre 8 : Les fonctions

I. Généralité

Les fonctions permettent de décomposer un problème complexe en plus petits composants simples réutilisables. Ces composants une fois interconnecté permettent de résoudre un problème complexe.

L'utilisation d'une fonction nécessite deux étapes :

- **La déclaration :** on définit le nom de la fonction, ses paramètres et les instructions à exécuter ;
- **L'appel :** on exécute la fonction pour des valeurs explicites des paramètres.

II. La déclaration d'une fonction

La syntaxe Python pour la définition d'une fonction est la suivante :

```
1. def nom_de_la_fonction(<parametres>):
2.     <bloc d'instructions>
```

Remarque :

- La définition d'une fonction commence par le mot-clé `def`;
- Le nom des fonctions doit respecter les mêmes règles que les variables. Les noms commençant par une majuscule seront réservés aux classes ;
- Les paramètres sont séparés par des virgules. Il peut ne pas y en avoir : dans ce cas, on conserve tout de même les parenthèses ;
- La première ligne se termine par un double point ;
- Le bloc `<bloc d'instructions>` est indenté. On l'appelle le corps de la fonction ;
- Lorsqu'il s'agit d'une fonction qui retourne un résultat, le mot-clé `return` est utilisé pour retourner une valeur.

III. L'appel d'une fonction

La syntaxe de l'appel d'une fonction est :

```
nom_de_la_fonction([<argument 1>, <argument 2>, …, <argument n>])
```

Rappels :

- Le type d'un paramètre sera le même que celui de l'argument qui aura été transmis à la fonction ;
- Le nom d'une variable que nous passons comme argument n'a rien à voir avec le nom du paramètre correspondant dans la fonction ;
- Lorsque nous transmettons une référence et que nous modifions la référence reçue en un autre élément, la connexion entre les paramètres transmis et reçus est rompue.

Exemple :

```
>>> def f(x, y):
...     return 2 * x + y
...
>>> f(0, 1)
1
>>> f(2, 1)
5
```

```
>>> def changement(a):
...     a = 10
...
>>> x = 2
>>> changement(x)
>>> print(x)
2
```

IV. Notions de variables locales et globales

En Python, les variables locales et globales sont liées à la visibilité des variables dans différentes parties de votre code.

1. Les variables locales

Une variable locale est déclarée à l'intérieur d'une fonction ou d'un bloc de code et n'est accessible qu'à l'intérieur de cette fonction ou de ce bloc. Sa durée de vie est limitée à l'exécution de la fonction ou du bloc.

Exemple :

Dans cet exemple, x est une variable déclarée dans le scope de la fonction `calcul`. Elle n'est donc pas disponible en dehors de ce scope.

```
>>> def calcul():
...     x = 42 * 2
...     print(x)
...
>>> calcul()
84
>>> print(x)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'x' is not defined
```

2. Les variables globales

Une variable globale est déclarée en dehors de toute fonction ou bloc et peut être accédée depuis n'importe où dans le programme. Sa durée de vie s'étend sur toute la durée d'exécution du programme.

```
1. >>> x = 42
2. >>> def calcul():
3. ...     x = 42 * 2
4. ...     print(x)
5. ...
6. >>> calcul()
7. 84
8. >>> print(x)
9. 42
```

Conventions :

- On utilise un underscore (par exemple `_ma_variable`) pour indiquer qu'une variable est protégée (accessible uniquement à l'intérieur de la classe ou du module).
- On utilise deux underscores (par exemple `__ma_variable`) pour indiquer qu'une variable est privée (accessible uniquement à l'intérieur de la classe).

3. Le mot clé global

Le mot-clé `global` est utilisé pour qualifier une variable comme étant globale dans un contexte où elle pourrait être locale. Son utilisation permet d'indiquer explicitement que vous utilisez une

```
 1. >>> x = 42
 2. >>> def calcul():
 3. ...     global x
 4. ...     x = 10
 5. ...     print(x)
 6. ...
 7. >>> calcul()
 8. 10
 9. >>> print(x)
10. 10
```

variable globale, ce qui évite toute confusion avec une variable locale du même nom.

V. Un peu plus sur la définition des fonctions

1. Valeur par défaut des paramètres

Python donne la possibilité de définir des valeurs par défaut pour chacun des paramètres. Cela permet de créer une fonction qui peut être appelée avec moins d'arguments que ceux présents dans sa définition.

Avertissement : Dans la définition d'une fonction, la déclaration des paramètres avec des valeurs par défaut doivent être placés après les paramètres sans valeurs par défaut.

```
>>> def salut(nom='toi'):
...     print(f"Salut {nom} !")
...
>>> salut('bae')
Salut bae !
>>> salut('Kate')
Salut Kate !
>>> salut()
Salut toi !
```

2. Les arguments nommés

Dans la plupart des langages de programmation, les arguments que l'on fournit lors de l'appel d'une fonction doivent placés exactement dans le même ordre que celui des paramètres qui leur correspondent dans la définition de la fonction.

Cependant, Python nous permet de passer les arguments dans n'importe quel ordre en précisant le nom du paramètre que nous souhaitons passer.

```
>>> def f(x, y):
...     return 2 * x + y
...
>>> f(2, 1)
5
>>> f(y = 1, x = 2)
5
```

NB :

- Dans un appel de fonction, les arguments nommés doivent suivre les arguments positionnés ;
- Tous les arguments nommés doivent correspondre à l'un des arguments acceptés par la fonction ;
- Quand un dernier paramètre formel est présent sous la forme `**name`, il reçoit un dictionnaire ;

VI. Arguments de longueur variable

En Python, les arguments de longueur variable permettent de créer des fonctions qui acceptent un nombre arbitraire d'arguments lors de leur appel.

1. Les arguments non nommés *args

Elle permet d'obtenir un nombre de paramètres variable sous forme d'une liste.

```
1. >>> def somme(x1, x2):
2. ...     return x1 + x2
3. ...
4. >>> somme(1, 2)
5. 3
```

Cette fonction **somme** ne peut additionner que deux nombres au maximum par appel.

```
 1. >>> def somme(*args):
 2. ...     print(args)
 3. ...     return sum(args)
 4. ...
 5. >>> somme(1, 2)
 6. (1, 2)
 7. 3
 8. >>> somme(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
 9. (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
10. 55
```

Cette fonction `somme` peut additionner un nombre arbitraire de nombres par appel.

2. Les arguments nommés **kwargs

Elle permet d'obtenir un nombre de paramètres variable sous forme d'un dictionnaire

Il est possible d'utiliser à la fois ***args** et ****kwargs** dans une même fonction. Dans ce cas ***args** collecte les arguments positionnels, tandis que ****kwargs** collecte les arguments de mots-clés.

```
1. >>> def affiche_infos(**kwargs):
2. ...     for cle, valeur in kwargs.items():
3. ...             print(f"{cle}: {valeur}")
4. ...
5. >>> affiche_infos(nom="Sandra", age = 18, ville="Lomé")
6. nom: Sandra
7. age: 18
8. ville: Lomé
```

VII. Fonctions anonymes

En Python, une fonction anonyme est une fonction définie sans nom. Contrairement aux fonctions normales qui utilisent le mot-clé `def`, les fonctions anonymes sont créées à l'aide du mot-clé `lambda`. C'est pourquoi elles sont également appelées fonctions `lambda`.

Une fonction lambda prend des arguments (ou paramètres) et renvoie une expression évaluée.

Syntaxe :

```
lambda <arguments>: <expression>
```

Exemple :

VIII. Fonctions récursives

Les fonctions récursives en Python sont utilisées pour décomposer une grande tâche en de plus petites en s'appelant de façon répétitive.

```
1. >>> addition = lambda a, b: a + b
2. >>> print(addition(2, 3))
3. 5
4. >>> nombres = [1, 2, 3, 4, 5, 6, 7, 8]
5. >>> nombres_pairs = list(filter(lambda x: x % 2 == 0, nombres))
6. >>> print(nombres_pairs)
7. [2, 4, 6, 8]
```

Une fonction récursive nécessite :

- **Un cas de base :** C'est le cas le plus simple. Elle permet d'arrêter l'exécution
- **Un ou plusieurs cas récursifs :** Elle consiste à l'appel de la fonction à elle-même qui conduit progressivement à la fonction au cas de base.

La méthode récursive en Python permet de décomposer un problème complexe en sous-problèmes plus simples qui peuvent être résolus de manière indépendante, puis d'assembler les solutions pour obtenir la solution globale.

Exemple : Calcul de la factorielle

```
1. >>> def factorielle(n):
2. ...     if n == 0:
3. ...         return 1
4. ...     else:
5. ...         return n * factorielle(n - 1)
6. ...
7. >>> print(factorielle(5))
8. 120
```

IX. Chaînes de documentation

Il convient que la première ligne d'instruction d'une fonction soit toujours courte et résume de manière concise l'utilité de l'objet. La convention veut que la ligne commence par une majuscule et se termine par un point.

S'il y a d'autres lignes dans la chaîne de documentation, la deuxième ligne devrait être vide, pour la séparer visuellement du reste de la description.

Exemple :

```
>>> def somme(*args):
...     """Calcul la somme des nombres.
...
...      Les nombres sont fournis comme arguments à longueur variable.
...     """
...     return sum(args)
...
>>> help(somme)
Help on function somme in module __main__:
somme(*args)
    Calcul la somme des nombres.

    Les nombres sont fournis comme arguments à longueur variable.
```

X. Les modules

Python vient par défaut avec des fonctions dites standards. Comme exemple, nous avons `print`, `len`, `input`, etc.

À côté de ces fonctions standard, nous pouvons créer nos propres fonctions et les regroupés dans un fichier afin de faciliter leurs réutilisations : Cela porte le nom de module.

Les modules sont donc des fichiers qui regroupent des ensembles de fonctions. En effet, il peut être commode de découper un programme important en plusieurs fichiers de taille modeste pour en faciliter la maintenance et le travail collectif.

N'importe quel fichier `.py` peut donc être appelé depuis un autre comme un module. Il peut contenir : du script, des fonctions, des classes, des constantes, etc.

1. Quelques fonctions standards

Comme exemple de modules standards, nous avons :

- **`math` :** Il fournit un accès aux fonctions mathématiques définies par la norme C
- **`os` :** Il fournit des fonctions pour interagir avec le système d'exploitation
- **`random` :** Il permet de générer des nombres aléatoires.
- **`time` :** Il fournit des fonctions pour travailler avec le temps et les dates.
- **`urlli` :** il permet de récupérer des données à partir d'Internet, d'ouvrir des fichiers distants, etc.

2. Importation et utilisation de module

L'instruction import est utilisée pour l'importation de modules.

- Voici quelques exemples avec la librairie **`math`** :

a. Importation de tout le contenu d'un module

```
>>> import math
>>> rc = sqrt(16)
>>> print(f"La racine carrée de 16 est {rc}")
La racine carrée de 16 est 4.0
```

b. Sélection de certains éléments

```
>>> from math import cos
>>> print(f"cos(2) = {cos(2)}")
cos(2) = -0.4161468365471424
```

3. Le module turtle

- Le module **`turtle`** en Python est un outil graphique qui permet de dessiner des formes en utilisant des instructions simples. Il est inspiré de la programmation Logo et permet de déplacer une "tortue" (souvent symbolisée par une flèche) sur l'écran.

Exemple : Dessin d'un carré

```
from turtle import *

for _ in range(4):
    forward(100)
    left(90)

done()
```

Chapitre 9 : Les fichiers

Les variables constituent un moyen pour stocker les données d'un programme en cours d'exécution. Cependant, une fois l'exécution du programme terminée, les zones mémoires dans la RAM appartenant au programme sont libérées, ce qui fait perdre les informations calculées ou manipulées par le programme.

Les fichiers sont un moyen de garder les informations à long terme, même une fois l'exécution du programme terminée.

La gestion de fichiers se résume généralement dans les trois étapes suivantes :

- Ouverture du fichier
- Traitements sur le fichier (Opérations de lecture, ou d'écriture).
- Fermeture du fichier.

I. Ouverture d'un fichier

1. La fonction open

La fonction open est utilisée pour ouvrir un fichier en Python. Elle renvoie un objet fichier et est le plus souvent utilisée avec deux arguments positionnels et un argument nommé.

Syntaxe :

```
open(<nom du fichier>, <mode>, <encodage>)
```

NB :

- Le premier argument est une chaîne contenant le nom du fichier.
- Le deuxième argument indique le mode d'ouverture du fichier. L'argument mode est optionnel, sa valeur par défaut est `'r'`
- L'encodage détermine comment les caractères textuels sont stockés dans le fichier sur le disque. Python utilise UTF-8 comme encodage par défaut pour les fichiers texte. L'UTF-8 est capable de représenter tous les caractères du standard Unicode.
- L'ajout d'un `'b'` au mode ouvre le fichier en mode binaire. Les données en mode binaire sont lues et écrites sous forme d'objets bytes.

2. Les modes d'ouvertures

Ce tableau résume les principaux modes d'ouverture en Python :

Mode	Description
`'r'`	Ouverture en lecture (par défaut). Le fichier doit exister.
`'w'`	Ouverture en écriture. Le fichier est créé s'il n'existe pas, sinon il est écrasé.
`'x'`	Création d'un nouveau fichier. Retourne une erreur si le fichier existe déjà.
`'a'`	Ouverture en mode ajout. Les données sont ajoutées à la fin du fichier s'il existe, sinon le fichier est créé.
`'+'`	Ouvrir un fichier pour la mise à jour (lecture et écriture)

En plus de ces modes de base, il existe des variantes qui permettent de combiner lecture et écriture :

- `'r+'` : Ouverture pour lire et écrire. Le fichier doit exister.
- `'w+'` : Ouverture pour écrire et lire. Le fichier est créé s'il n'existe pas, sinon il est écrasé.
- `'a+'` : Ouverture pour ajouter et lire. Les données sont ajoutées à la fin du fichier s'il existe, sinon le fichier est créé.

Pour les fichiers binaires, il faut ajouter `'b'` au mode comme **`'rb'`**, **`'wb'`**, **`'ab'`**, **`'rb+'`**, **`'wb+'`**, **`'ab+'`**.

Exemple :

```
f = open('etudiants.txt', 'r', encoding='utf-8')
```

II. Traitement sur un fichier

1. Lecture

Pour lire depuis fichier, il doit exister et être impérativement ouvert en mode `"r"`, sinon une exception de type **`FileNotFoundError`** sera levée.

a. La méthode read

La méthode **`read(size)`** en Python est utilisée pour lire le contenu d'un fichier ouvert en lecture. Par défaut, il lira et retournera tout le contenu du fichier jusqu'à la fin. Dans le cas où `size` est spécifié **`read()`** lira jusqu'à ce nombre d'octets à partir de la position actuelle du curseur dans le fichier.

Exemple :

```
1. >>> f = open('texte.txt', 'r', encoding='utf-8')
2. >>> f.read(162)
3. "Il est l'heure, fini l'heure de danser\nDanse, t'inquiète pas tu vas
danser\nBalance-toi, mais tu vas te faire balancer\nDéfonce-toi, mais tu vas
te faire défoncer\n\n"
4. >>> f.read()
5. "Tu aimerais faire ta fête\nTa mère veut te la faire aussi, ta fête\nLe
juge voudrait te faire ta fête\nTout le monde te fera aussi ta fête\n\nTu sors
trop, du moins c'est ce qu'ils …"
```

b. La méthode readline

La méthode **`readline`** effectue la lecture d'un fichier ligne par ligne à partir de la position actuelle du curseur dans le fichier. Elle retourne la ligne lue sous forme de chaîne de caractères, y compris le caractère de nouvelle ligne (`'\n'`).

Dans le cas où un argument `n` est fourni, **`readline(n)`** lira au maximum `n` caractères de la ligne courante.

```
1. >>> f = open('texte.txt', 'r', encoding='utf-8')
2. >>> f.readline()
3. "Il est l'heure, fini l'heure de danser\n"
4. >>> f.readline()
5. "Danse, t'inquiète pas tu vas danser\n"
```

— **Lecture du fichier à l'aide d'une boucle for**

```
 1. >>> f = open('goat.txt', 'r', encoding='utf-8')
 2. >>> for line in f:
 3. ...     print(line, end='')
 4. ...
 5. Low-key
 6. All of them don dey notice
 7. I'm not joking
 8. All my guys dem, dem know me
 9. Since '06
10. I've been writin' this story
11. I no be OG
12. But, if you ask dem, dem know me
```

c. La méthode readlines

La méthode **readlines** effectue la lecture de toutes les lignes du fichier et les retourne sous forme de liste de chaînes de caractères. Chaque élément de la liste correspond à une ligne du fichier.

Dans le cas où un argument **n** est fourni, **readlines(n)** lira les lignes du fichier jusqu'à ce que le nombre total de caractères lus atteigne **n** ou jusqu'à la fin du fichier, en s'assurant qu'une ligne complète est toujours retournée.

```
1. >>> f = open('avoir.txt', 'r', encoding='utf-8')
2. >>> f.readlines(260)
3. ['Je jouerais du même harmonica\n', 'Je verrais le même arbre là-bas\n',
'Serais-je heureux sans toi? Pourquoi pas?\n', "Rien que d'y penser ça me glace à
chaque fois\n", '\n', "Si je t'avais pas\n", "Si je t'avais pas\n", 'Que serais-
je, où ça?\n', "Ma maison c'est là\n", 'Exactement dans tes bras\n', '\n',
"J'aurais des enfants, mais pas ceux-là\n"]
```

2. Ecriture d'un fichier

Pour écrire dans un fichier, il doit être ouvert avec le mode `"w"`.

Attention :

- Si le fichier n'existe pas il sera créé, s'il existe il sera tronqué ;
- Il est important de fermer le fichier avec la méthode `close()` pour s'assurer que les données sont bien écrites sur le disque.

Exemple :

```
 1. >>> f = open('test.txt', 'w', encoding='utf-8')
 2. >>> f.write("I'm not afraid (I'm not afraid)\n")
 3. 32
 4. >>> f.write("Yeah\n")
 5. 5
 6. >>> f.write("To take a stand (to take a stand)\n")
 7. 34
 8. >>> f.write("It's been a ride\n")
 9. 17
10. >>> f.close()
11. >>> f = open('test.txt', 'r', encoding='utf-8')
12. >>> f.readlines()
13. ["I'm not afraid (I'm not afraid)\n", 'Yeah\n', 'To take a stand
(to take a stand)\n', "It's been a ride\n"]
```

III. Fermeture d'un fichier

Il est important de fermer le fichier avec la méthode **`close`**() pour s'assurer que les données sont bien écrites sur le disque.

Exemple :

```
>>> f.close()
```

IV. L'instruction with

L'instruction **`with`** est utilisée comme gestionnaire de contexte qui permet de simplifier certains modèles courants de gestion des ressources. Elle est souvent utilisée avec des opérations qui nécessitent de mettre en place une séquence d'initialisation et de finalisation, comme l'ouverture et la fermeture de fichiers. Pour écarter le risque de laisser un fichier ouvert, nous pouvons utiliser l'instruction **`with`** qui s'assurera de fermer le fichier même si une exception est levée à l'intérieur du bloc.

Exemple :

```
1. with open('hello.txt', 'w') as f:
2.     f.write('Bonjour, le monde!')
```

Chapitre 10 : Les exceptions

I. Généralités

1. Définition

Les exceptions en Python sont des événements qui se produisent lors de l'exécution d'un programme et qui interrompent le flux normal des instructions. Elles sont utilisées pour gérer les erreurs qui peuvent survenir pendant l'exécution d'un script.

Python dispose de deux sortes d'erreurs :

Erreurs de syntaxe : Ce sont des erreurs d'analyse du code, souvent dues à des erreurs d'inattention ou à une mauvaise connaissance de la syntaxe de Python.

Exceptions : Une instruction syntaxiquement correcte, peut générer une erreur lors de son exécution. Ces erreurs détectées durant l'exécution sont appelées des exceptions.

2. Quelques exceptions

Voici quelques-unes des exceptions standard les plus couramment rencontrées :

- **`IndexError` :** Levée lorsqu'un index n'est pas trouvé dans une séquence.
- **`ImportError` :** Levée lorsqu'un module importé ne peut être trouvé.
- **`IOError` :** Levée lorsqu'une opération d'entrée/sortie échoue.
- **`ZeroDivisionError` :** Levée lorsqu'une division ou un modulo par zéro est effectué.
- **`TypeError` :** Levée lorsqu'une opération ou une fonction est appliquée à un objet d'un type inapproprié.
- **`NameError` :** Levée lorsqu'une variable locale ou globale n'est pas trouvée.
- **`KeyError` :** Levée lorsqu'une clé n'est pas trouvée dans un dictionnaire.
- **`ValueError` :** Levée lorsqu'une fonction reçoit un argument de bon type mais de valeur inappropriée.
- **`AttributeError` :** Levée lorsqu'un attribut de référence ou une affectation échoue.

II. La gestion des erreurs

Par défaut, lorsqu'une exception est levée dans un programme en cours d'exécution, son exécution est interrompue quand l'erreur n'est pas capturée. Dans de nombreux cas, il est possible de prévoir certaines des erreurs qui risquent de se produire à tel ou tel endroit du programme, et d'inclure à cet endroit des instructions particulières, qui seront activées seulement si ces erreurs se produisent.

La capture des exceptions en Python est un mécanisme essentiel pour la gestion des erreurs et la robustesse des programmes. Elle utilise les mots-clés **`try`** et **`except`** pour intercepter les erreurs qui se produisent pendant l'exécution du code.

1. Le bloc d'instruction try..except

Syntaxe :

```
try:
    # Code pouvant générer une exception
except <nom de l'exception>:
    # Code à exécuter en cas d'exception
else:
    # Code à exécuter en cas de non exception
finally:
    # Code à exécuter dans tous les cas
```

Exemple :

```
>>> def division(a, b):
...     try:
...         d = a / b
...         print(d)
...     except ZeroDivisionError:
...         print('Division par 0 détectée')
...
>>> division(4, 2)
2.0
>>> division(4, 0)
Division par 0 détectée
```

III. Lever d'une exception

Un programme Python peut lever une exception lorsqu'il détecte une erreur, en utilisant l'instruction raise.

```
>>> def prix_total(qte):
...     if qte <= 0:
...         raise Exception("La quantité spécifiée est invalide.")
...     else:
...         return qte * 5000
...
>>> prix_total(2)
10000
>>> prix_total(0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 3, in prix_total
Exception: La quantité spécifiée est invalide.
```

IV. Contrôles de saisies

Les contrôles de saisies grâce aux exceptions en Python permettent de valider les entrées utilisateur et de s'assurer qu'elles correspondent aux attentes du programme.

Exemple :

```
>>> while True:
...     try:
...         # Demande à l'utilisateur de saisir un nombre
...         nombre = int(input("Veuillez saisir un nombre : "))
...         # Si la saisie est correcte, le programme continue
...         print(f"Vous avez saisi le nombre {nombre}.")
...         break  # Sortie de la boucle si tout est correct
...     except ValueError:
...         # Si la saisie n'est pas un nombre, informe l'utilisateur
...         print("Ce n'est pas un nombre valide. Veuillez réessayer.")
...
Veuillez saisir un nombre : aze
Ce n'est pas un nombre valide. Veuillez réessayer.
Veuillez saisir un nombre : 10
Vous avez saisi le nombre 10.
```

V. Les assertions

Les assertions en Python sont une manière de vérifier que certaines conditions sont vraies à un moment donné dans un programme. Elles sont utilisées principalement pour le débogage, car elles permettent de s'assurer que le code fonctionne comme prévu.

L'instruction **`assert`** en Python est utilisée pour définir une condition de débogage qui doit toujours être vraie à un certain point du programme. Si la condition est fausse, Python lève une **`AssertionError`**.

Syntaxe :

```
assert condition [, <message>]
```

Exemple 1 : Utilisation simple

```
1. >>> qte = 0
2. >>> assert qte > 0, "La quantité doit être > 0"
3. Traceback (most recent call last):
4.   File "<stdin>", line 1, in <module>
5. AssertionError: La quantité doit être > 0
```

Exemple 2 : Utilisation dans une fonction

```
 1. >>> def appliquer_reduction(prix, reduction):
 2. ...     prix_reduit = prix - (prix * reduction)
 3. ...     assert 0 <= prix_reduit <= prix, "Le prix réduit ne peut être être
inférieurs à 0 ni depasser le prix d'origine du produit."
 4. ...     return prix_reduit
 5. ...
 6. >>> print(appliquer_reduction(10000, 20/100)) # Reduction de 20%
 7. 8000.0
 8. >>> print(appliquer_reduction(10000, 101/100)) # Reduction de 101%
 9. Traceback (most recent call last):
10.   File "<stdin>", line 1, in <module>
11.   File "<stdin>", line 3, in appliquer_reduction
12. AssertionError: Le prix réduit ne peut être être inférieurs à 0 ni depasser le
prix d'origine du produit.
```

PARTIE II: LES STRUCTURES DE DONNEES EN PYTHON

Chapitre 1 : Présentation et intérêt des structures de données

Le développement de système d'informations implique la manipulation de données devant être organisées de manière optimisée afin de faciliter les opérations comme la recherche, l'ajout de nouvelles informations ou la suppression. Les structures de données nous permettent d'atteindre cet objectif.

L'étude des structures de données revêt une importance cruciale dans le domaine de l'informatique et du développement logiciel. Leur principal objectif est d'organiser et de stocker des données de manière efficace afin de faciliter leur manipulation, leur recherche et leur accès.

I. Quelques rôles essentiels des structures de données :

- **Organisation des données :** Les structures de données permettent d'organiser les données de manière logique, cohérente et efficiente, ce qui facilite leur gestion et leur utilisation dans les programmes informatiques.
- **Stockage efficace :** Elles offrent des mécanismes pour stocker des données en mémoire ou sur un support de stockage de manière efficace à travers les fichiers, en minimisant l'utilisation de ressources telles que la mémoire vive ou le disque dur.
- **Accès rapide :** Le choix des structures de données ayant des influences sur le temps d'accès, en choisissant la bonne structure de données, il est possible d'accéder rapidement aux données nécessaires, ce qui améliore les performances des algorithmes et des applications.
- **Manipulation et traitement des données :** Les structures de données fournissent des opérations et des méthodes pour manipuler et traiter les données, telles que l'ajout, la suppression, la recherche, le tri, etc.
- **Optimisation des algorithmes :** En utilisant les bonnes structures de données, il est possible d'optimiser les algorithmes pour des opérations spécifiques, ce qui améliore les performances et réduit la complexité temporelle et spatiale.

II. Implémentation des structures de données

Les concepts relatifs aux structures de données sont universels. Néanmoins, chaque langage dispose de sa propre implémentation. Dans le cadre de ce cours, nous utiliserons le langage de

programmation Python comme langage d'implémentation des structures de données qui seront étudiés.

1. Les structures de données natives du langage Python

Python offre plusieurs types de structures de données natifs qui sont essentiels pour la programmation. Citons quelques-unes de ces structures :

- **Les listes (list):** Une collection ordonnée et modifiable d'éléments. Les listes permettent d'ajouter, de supprimer et de manipuler facilement leurs éléments ;
- **Les tableaux (array) :** Les tableaux sont de type séquence et se comportent de manière similaire aux listes, sauf que les types d'objets qu'ils stockent sont limités ;
- **Tuple (tuple) :** Une séquence immuable et hétérogène d'éléments. Les tuples sont utiles pour stocker des données qui ne doivent pas être modifiées ;
- **Ensemble (set) :** Une collection non ordonnée d'éléments uniques. Les ensembles permettent de réaliser des opérations ensemblistes comme l'union, l'intersection ou la différence ;
- **Dictionnaire (dict) :** Une collection non ordonnée de paires clé-valeur. Les dictionnaires permettent d'accéder rapidement aux valeurs à l'aide de leurs clés.

2. Les autres structures de données

En dehors de structures de données natives, il est possible au développeur d'implémenter d'autres structures pour des besoins spécifiques. Nous avons :

- Les Listes chaînées (Linked List)
- Les Piles et files (Stack et Queue)
- Les Arbres (Tree)
- Les Arbres binaires de recherche (Binary Search Tree)

Chapitre 2 : Les listes

I. Généralité

Une liste (`list`) est une collection ordonnée et modifiable d'éléments. Les listes permettent d'ajouter, de supprimer et de manipuler facilement. Elle peut contenir des valeurs de différents types tels que des nombres, des chaînes de caractères, des booléens, etc.

Les éléments d'une liste sont indexés à partir de zéro (le premier élément a l'indice `0`, le deuxième a l'indice `1`, et ainsi de suite).

II. Déclaration d'une liste

Les listes sont déclarées à l'aide de l'opérateur d'indiçage (`[]`). On peut également l'initialiser avec une séquence de valeur séparée par des virgules.

Syntaxe :

```
<nom de variable> = []
```

Exemple :

```
1. >>> nombres = [1, 2, 5, 7, 10]
2. >>> print(nombres)
3. [1, 2, 5, 7, 10]
4. >>> mixtes = [1, 'a', 2, 'b', 3, 'c']
5. >>> print(mixtes)
6. [1, 'a', 2, 'b', 3, 'c']
```

NB :

- Une seule liste peut contenir des types différents tels que des entiers, des chaînes, ainsi que des objets ;
- Les listes sont également très utiles pour l'implémentation de piles et de files d'attente ;
- Les listes sont modifiables et peuvent donc être modifiées même après leur création.

III. Accès aux éléments d'une liste

Tout comme les chaines de caractères, les indices des éléments d'une liste de n éléments commencent à 0 et se terminent à $n-1$. Pour accéder au i-ème élément d'une liste, on utilise la

```
liste[i]
```

Syntaxe :

Les indices d'une liste sont similaires à ceux des caractères d'une chaine de caractères étudié plus haut dans ce court. Les techniques de silicing s'appliquent également aux listes.

```
>>> liste = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> # Afficher un élément sur deux
>>> print(liste[::2])
[1, 3, 5, 7, 9]
>>>
>>> # Afficher les 3 premiers éléments d'une liste
>>> print(liste[:3])
[1, 2, 3]
>>>
>>> # Afficher les 4 derniers éléments d'une liste
>>> print(liste[-4:])
[7, 8, 9, 10]
>>>
>>> # Extraire les éléments en partant de l'indice 2
(inclus)
>>> # jusqu'à l'indice 8 (exclus)
>>> # et en utilisant une fréquence de 2
>>> print(liste[2:8:2])
[3, 5, 7]
```

IV. Opérations sur les listes

1. Manipulations de listes

a. Affectation

A la différence des chaines de caractères qui sont un type de donnée non modifiable (immutables), les listes sont mutables. Il est donc possible de changer les éléments individuels d'une liste après sa déclaration.

Exemple :

```
>>> nombres = [2, 4, 5]
>>> nombres[1] = nombres[1] * 2
>>> nombres
[2, 8, 5]
```

b. Concaténation

L'opérateur + peut être utilisé pour fusionner deux listes.

```
>>> fruits = ['orange','citron']
>>> legumes = ['poireau','oignon','tomate']
>>> fruits + legumes
['orange', 'citron', 'poireau', 'oignon', 'tomate']
```

c. La répétition

L'opérateur * peut être utilisé pour répéter les éléments d'une liste.

```
1. >>> liste = [10]
2. >>> liste = liste * 7
3. >>> liste
4. [10, 10, 10, 10, 10, 10, 10]
```

d. La suppression d'éléments

Nous pouvons supprimer un ou plusieurs éléments d'une liste partir de leurs index grâce à la fonction `del`.

```
1. >>> del(liste[0])
2. >>> liste
3. [2, 3, 4, 5, 6, 7, 8, 9, 10]
4. >>> del liste[0:3]
5. >>> liste
6. [5, 6, 7, 8, 9, 10]
```

e. Remplacement d'éléments

Il est possible de remplacer une partie d'une liste après sa déclaration.

```
1. >>> liste = [0, 1, 2, 3, 4, 5, 6]
2. >>> liste = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
3. >>> liste[2:5] = []
4. >>> liste
5. [0, 1, 5, 6, 7, 8, 9, 10]
6. >>> liste[3:8] = [10]
7. >>> liste
8. [0, 1, 5, 10]
```

f. Copie de liste

Lorsqu'une variable contenant une liste est affectée à une autre variable, le contenu de la première variable est copié dans la seconde variable. Les deux variables partagent désormais la même référence. Ceci peut notamment être vérifié grâce à la fonction **`id()`**.

Afin de copier le contenu d'une liste sans toutefois garder la même référence que l'objet source, les fonctions `copy` et **`list`** peut être utilisées. Dans le cas où la liste à copier contient elle-même d'autres listes, la fonction **`deepcopy`** est nécessaire.

Exemple :

```
>>> # Copie par affectation directe
>>> liste1 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> liste2 = liste1
>>> liste2
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> id(liste1)
1765682729408
>>> id(liste2)
1765682729408
>>> # Copie grâce à la fonction copy
>>> liste2 = liste1.copy()
>>> liste2
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> id(liste1)
1765682729408
>>> id(liste2)
1765682685248
>>> # Copie grâce à la fonction list
>>> liste2 = list(liste1)
>>> id(liste2)
1765679233728
>>> id(liste1)
1765682729408
>>> # Copie à l'aide du slicing
>>> liste2 = liste1[:]
>>> id(liste1)
1765682729408
>>> id(liste2)
1765682685248
```

2. Quelques méthodes utilisables sur les listes

Les listes étant des objets à part entière, un certain nombre de méthodes utiles sont disponibles après l'opérateur point sur un objet de type `list`. La liste exhaustive de ces méthodes peut être trouvée grâce l'instruction `help(list)`.

a. La méthode index

Retourne l'index de première occurrence. Les index de début et de fin ne sont pas des paramètres nécessaires.

```
1. >>> liste = ['a', 'b', 'a', 'a', 'c']
2. >>> liste.index('c')
3. 4
4. >>> liste.index('a')
5. 0
```

b. La méthode sum

Elle permet de calculer la somme de tous les éléments de la liste.

```
1. >>> liste = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
2. >>> sum(liste)
3. 55
```

c. La méthode count

Elle permet de connaitre le nombre d'éléments dans une liste.

```
1. >>> liste = ['a', 'b', 'a', 'a', 'c']
2. >>> liste.count('a')
3. 3
```

d. La méthode append

Elle permet d'ajouter un élément à la fin d'une liste.

```
1. >>> liste = ['a', 'b', 'a', 'a', 'c']
2. >>> liste
3. ['a', 'b', 'a', 'a', 'c']
4. >>> liste.append('z')
5. >>> liste
6. ['a', 'b', 'a', 'a', 'c', 'z']
```

e. La méthode clear

Cette fonction permet d'effacer tous les éléments de la liste. Après cette opération, la liste devient vide.

```
1. >>> liste = ['a', 'b', 'a', 'a', 'c']
2. >>> liste.clear()
3. >>> liste
4. []
```

f. La méthode insert

Elle permet d'insérer un objet après un index passé en paramètre.

```
1. >>> liste = ['a', 'b', 'a', 'a', 'c']
2. >>> liste.insert(2, 'z')
3. >>> liste
4. ['a', 'b', 'z', 'a', 'a', 'c']
```

g. La méthode pop

Cette fonction supprime et retourne un objet à un index par défaut le dernier élément. Elle soulève une exception de type `IndexError` si la liste est vide ou l'index est en dehors de la plage des index de la liste.

```
1. >>> liste = ['a', 'b', 'a', 'a', 'c']
2. >>> liste.pop()
3. 'c'
4. >>> liste
5. ['a', 'b', 'a', 'a']
6. >>> liste.pop(2)
7. 'a'
8. >>> liste
9. ['a', 'b', 'a']
```

h. La méthode remove

Cette fonction supprime la première occurrence d'une. Elle soulève une exception de type `ValueError` si la valeur n'est pas présente dans la liste.

```
1. >>> liste = ['a', 'b', 'a', 'a', 'c']
2. >>> liste.remove('a')
3. >>> liste
4. ['b', 'a', 'a', 'c']
5. >>> liste.remove('a')
6. >>> liste
7. ['b', 'a', 'c']
```

i. La méthode reverse

Cette fonction renverse l'ordre des éléments dans la liste. La liste directement est modifiée et None est retourné.

```
1. >>> liste = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
2. >>> liste.reverse()
3. >>> liste
4. [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

j. La méthode sort

Tri la liste dans l'ordre croissant. La liste directement est modifiée et `None` est retourné. L'ordre des éléments étant maintenus, il s'agit d'un tri stable. Le paramètre reverse peut-être précisé pour contrôler le type de tri.

```
>>> # Tri par ordre croissant
>>> liste = [10, 8, 7, 9, 5, 6, 3, 2, 4, 1]
>>> liste
[10, 8, 7, 9, 5, 6, 3, 2, 4, 1]
>>> liste.sort()
>>> liste
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> # Tri par ordre décroissant
>>> liste = [10, 8, 7, 9, 5, 6, 3, 2, 4, 1]
>>> liste.sort(reverse=True)
>>> liste
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

3. Quelques fonctions utilisables sur les listes

a. Les fonctions min et max

Elles permettent respectivement de connaitre le minimum et le maximum d'une liste.

Exemple :

```
>>> liste = [10, 8, 7, 9, 5, 6, 3, 2, 4]
>>> min(liste)
2
>>> max(liste)
10
```

b. Les fonctions len

Elle permet de connaitre la taille d'une liste.

```
>>> liste = ['a', 'b', 'a', 'a', 'c']
>>> len(liste)
5
```

c. La fonction list

La fonction list permet de construire une liste à partir d'un itérable.

```
>>> nombres = list(range(11))
>>> nombres
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

V. Liste de liste

Il est possible qu'une liste puisse contenir une autre liste qui a son tour peut contenir une autre liste et ainsi de suite.

```
>>> enclos1 = ["girafe", 4]
>>> enclos2 = ["tigre", 2]
>>> enclos3 = ["singe", 5]
>>> zoo = [enclos1, enclos2, enclos3]
>>> zoo
[['girafe', 4], ['tigre', 2], ['singe', 5]]
>>> for enclos in zoo:
...     print(f"Il y a {enclos[1]} {enclos[0]} dans l'enclos")
...
Il y a 4 girafes dans l'enclos
Il y a 2 tigres dans l'enclos
Il y a 5 singes dans l'enclos
>>> for idx, enclos in enumerate(zoo):
...     print(f"Il y a {enclos[1]} {enclos[0]} dans l'enclos {idx+1}")
...
Il y a 4 girafes dans l'enclos 1
Il y a 2 tigres dans l'enclos 2
Il y a 5 singes dans l'enclos 3
>>> for i in range(len(zoo)):
...     print(f"Il y a {zoo[i][1]} {zoo[i][0]} dans l'enclos {i+1}")
...
Il y a 4 girafes dans l'enclos 1
Il y a 2 tigres dans l'enclos 2
Il y a 5 singes dans l'enclos 3
```

VI. Les tableaux de type array

En Python, le module **`array`** permet de créer des tableaux (ou **`arrays`**) de manière efficace pour stocker des valeurs numériques. Les tableaux sont de type séquence et se comportent de manière similaire aux listes, sauf que les types d'objets qu'ils stockent sont limités.

Le type d'objet est spécifié lors de la création du tableau en utilisant un code de type, qui est un caractère unique.

Quelques codes de type courants :

- `'b'` : entier signé sur `1` octet.
- `'f'` : nombre à virgule flottante sur `4` octets.
- `'d'` : nombre à virgule flottante sur `8` octets.

Syntaxe :

```
mon_tableau = array.array(typecode, [valeur1, valeur2, ...])
```

Exemple :

```
>>> import array
>>> nombres = array.array('i', [10, 20, 30, 40])
>>> nombres.append(50)
>>> for nombre in nombres:
...     print(nombre)
...
10
20
30
40
50
```

VII. Les listes en compréhension (List Comprehension)

Les listes en compréhension constituent un moyen élégant et simple permettant de créer des listes en Python. Elles sont définies comme un ensemble en mathématiques et permettent de générer une nouvelle liste en une seule ligne d'instruction, en utilisant une boucle for avec ou sans condition if.

Syntaxe simple :

```
L = [<expr> for <it> in <list>]
```

NB :

- Dans la paire de crochets, les mots-clefs `for` et `in` sont obligatoires ;
- `<list>` une liste ou un itérable ;
- `<it>` est un itérateur qui parcourt la liste `<list>` ; `<it>` est appelé variable de contrôle de la liste en compréhension ;
- `<expr>` est une expression qui dépend en général de `<it>` et dont la valeur est placée dans `L`.

1. Listes en compréhension à partir d'un itérable

La notation $\{ f(x) | x \in liste \}$ définie une fonction f pour chaque élément x appartenant à une à une certaine liste (ou ensemble). Cette notation peut être écrite en python comme :

```
[f(x) for x in liste]
```

Exemple :

```
>>> liste = list(range(10))
>>> liste
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> carre_nombres = [n**2 for n in liste]
>>> carre_nombres
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

2. Listes en compréhension et la clause if

La notation $\{ f(x) | x \in liste \text{ tel que } Condition(x) \}$ per permet d'ajouter à la liste L les éléments x de liste respectant la condition définie. Elle peut être écrite comme :

```
L = [f(x) for x in liste if Condition(x)]
```

Exemple :

```
>>> n = 120
>>> diviseurs = [i for i in range(1, 100) if n % i == 0]
>>> diviseurs
[1, 2, 3, 4, 5, 6, 8, 10, 12, 15, 20, 24, 30, 40, 60]
```

3. Listes en compréhension imbriquées

Il est possible d'imbriquer les listes en compréhension. Dans ce cas, l'expression est évaluée de la droite vers la gauche.

```
>>> couples = [x+y for x in "AB" for y in "123"]
>>> couples
['A1', 'A2', 'A3', 'B1', 'B2', 'B3']
>>> grille = [[(x, y) for y in range(3)] for x in range(3)]
>>> grille
[[(0, 0), (0, 1), (0, 2)], [(1, 0), (1, 1), (1, 2)], [(2, 0), (2, 1), (2, 2)]]
```

I. Généralité

Selon Wikipédia, "un ensemble ou set est un type abstrait qui peut stocker certaines valeurs, sans ordre particulier, et sans répétition. Il s'agit d'une mise en œuvre informatique de la notion mathématique d'ensemble fini."[2]

En Python, les ensembles sont définis par le mot "`set()` et ne peut pas contenir de doublons.

II. Création d'ensemble

1. Création d'un set vide

Syntaxe :

```
s = set()
```

2. Création d'un à partir d'un itérable

Exemple :

```
1. >>> lettres = {'a', 'b', 'c', 'c', 'a'}
2. >>> lettres
3. {'a', 'c', 'b'}
4. >>> lettres = set(['a', 'b', 'c', 'c', 'a'])
5. >>> lettres
6. {'a', 'c', 'b'}
```

III. Manipulation des éléments d'un set

1. Ajout d'un ou de plusieurs éléments

Les sets étant modifiable, on peut ajouter des éléments après sa création.

```
1. >>> lettres = set(['a', 'b', 'c', 'd'])
2. >>> lettres.add('e')
3. >>> lettres
4. {'d', 'a', 'b', 'e', 'c'}
5. >>> lettres.update('e', 'f', 'g')
6. >>> lettres
7. {'g', 'd', 'a', 'f', 'b', 'e', 'c'}
```

[2] https://fr.wikipedia.org/wiki/Ensemble_(informatique)

2. Recherche de membre

L'opérateur d'appartenance permet de vérifier si un élément se trouve ou non dans un set.

```
1. >>> lettres = set(['a', 'b', 'c', 'd'])
2. >>> 'a' in lettres
3. True
4. >>> 'z' in lettres
5. False
```

3. Retrait de membre d'un set

Python propose les fonctions pop, `remove`, `discard` et `clear` pour retirer un ou plusieurs membres d'un `set`.

a. La méthode pop

Elle retire un élément au hasard du set elle soulève une exception du type `KeyError` si le set est vide.

```
1. >>> lettres = set(['a', 'b', 'c', 'd'])
2. >>> lettres.pop()
3. 'd'
4. >>> lettres
5. {'a', 'c', 'b'}
```

b. La méthode remove

Elle retire un membre du set. Elle soulève une exception du type `KeyError` si l'élément n'est pas membre du set.

```
1. >>> lettres = set(['a', 'b', 'c', 'd'])
2. >>> lettres.remove('a')
3. >>> lettres
4. {'d', 'c', 'b'}
5. >>> lettres.remove('z')
6. Traceback (most recent call last):
7.   File "<stdin>", line 1, in <module>
8. KeyError: 'z'
```

c. La méthode discard

Elle fonctionne de la même manière que la méthode `remove` sauf qu'elle ne retourne pas d'exception dans le cas où l'élément à supprimer n'est pas membre du set.

d. La méthode clear

Retire tous les éléments d'un set.

```
1. >>> lettres = set(['a', 'b', 'c', 'd'])
2. >>> lettres.clear()
3. >>> lettres
4. set()
```

IV. Opération sur les ensembles

1. L'intersection ou AND

L'intersection de deux ensembles contient tous les éléments qui appartiennent aux deux ensembles. En Python, nous pouvons utiliser l'opérateur & ou la fonction intersection pour effectuer l'intersection de deux ensembles.

```
1. >>> a = {1, 2, 3}
2. >>> b = {3, 4, 5}
3. >>> a.intersection(b)
4. {3}
5. >>> a & b
6. {3}
```

2. L'union ou OR

L'union de deux ensembles contient tous les éléments qui appartiennent à l'un ou l'autre ensemble. L'opérateur | ou la fonction union peut être utiliser pour effectuer l'union de deux ensembles.

```
1. >>> a = {1, 2, 3}
2. >>> b = {3, 4, 5}
3. >>> a | b
4. {1, 2, 3, 4, 5}
5. >>> a.union(b)
6. {1, 2, 3, 4, 5}
```

3. La différence

La différence entre deux ensembles contient tous les éléments qui appartiennent à l'ensemble de départ mais pas à l'autre ensemble. L'opérateur - ou la fonction différence peut être utilisé

pour effectuer la différence entre deux ensembles. La fonction `difference_update` met à jour le set original en retirant les éléments non voulus.

```
1. >>> a = {1, 2, 3}
2. >>> b = {3, 4, 5}
3. >>> a - b
4. {1, 2}
```

4. La différence symétrique

Elle retourne un set contenant tous les éléments de deux sets sauf les éléments présents dans les deux sets.

```
1. >>> a = {1, 2, 3}
2. >>> b = {3, 4, 5}
3. >>> a.symmetric_difference(b)
4. {1, 2, 4, 5}
```

V. Les frozensets

Un `frozenset` (ensemble figé) se comporte comme un ensemble, sauf qu'il est immutable, c'est-à-dire qu'une fois créé, on ne peut pas le mettre à jour. Il dispose donc des mêmes fonctions que le type `"set"`, mais sans `"add"`, `"update"`, `"pop"`, `"remove"`, `"discard"` et `"clear"`.

De plus, ils sont hachables, ce qui leur permet de faire partie d'ensembles.

```
1. >>> fs = frozenset([1, 2, 3])
2. >>> fs.add(4)
3. Traceback (most recent call last):
4.   File "<stdin>", line 1, in <module>
5. AttributeError: 'frozenset' object has no attribute 'add'
```

I. Définition

Les tuples (« n-uplets » en français) sont des collections ordonnées finies d'objets. Ils sont semblables aux listes c'est-à-dire ils sont itérables, ordonnés et indexables, mais contrairement aux listes ils sont immutables. Ils ne peuvent pas être modifie après leur déclaration.

Les tuples sont souvent utilisés pour stocker des collections d'éléments hétérogènes.

II. Création d'un tuple

1. Création d'un tuple vide

```
1. >>> t = tuple()
2. >>> type(t)
3. <class 'tuple'>
```

2. Création d'un tuple à partir d'un itérable

```
1. >>> t = tuple('Bonjour !')
2. >>> t
3. ('B', 'o', 'n', 'j', 'o', 'u', 'r', ' ', '!')
4. >>> t = tuple([1, 2, 3, 4])
5. >>> t
6. (1, 2, 3, 4)
7. >>> t = (2, )
8. >>> t
9. (2,)
```

III. Opérations sur les tuples

Un tuple est essentiellement une liste immuable. En conséquence, la plupart des opérations supportées par les listes sont également supportées par les tuples.

1. Opérateurs + et *

Les opérateurs + et * des tuples fonctionnent comme pour les listes (concaténation et duplication) :

```
1. >>> t1 = (1, 2)
2. >>> t2 = (3, 4)
3. >>> t1 + t2
4. (1, 2, 3, 4)
5. >>> t1 * 2
6. (1, 2, 1, 2)
```

2. Opérateur d'indiçage

On peut accéder à un élément ou à des tranches d'éléments à l'aide de l'opérateur [].

```
1. >>> t = (1, 2, 3, 4, 5, 6, 7, 8)
2. >>> t[0]
3. 1
4. >>> t[-1]
5. 8
6. >>> t[::-1]
7. (8, 7, 6, 5, 4, 3, 2, 1)
```

3. Operateurs d'appartenance

Ils sont utilisés pour vérifier si un élément est présent dans un objet itérable, comme un tuple.

```
1. >>> t = (1, 2, 3, 4, 5, 6, 7, 8)
2. >>> 4 in t
3. True
4. >>> 10 in t
5. False
6. >>> 10 not in t
7. True
```

4. Comparaison de tuples

La comparaison de tuples en Python se fait de manière lexicographique, c'est-à-dire que les éléments sont comparés un par un, en commençant par le premier élément de chaque tuple.

La règle se résume comme suit :

- Si le premier élément de deux tuples est différent, le tuple avec le plus petit élément est considéré comme inférieur.
- Si les premiers éléments sont égaux, Python compare le second élément, et ainsi de suite.
- Si tous les éléments correspondants sont égaux, les tuples sont considérés comme égaux.
- Si un tuple est plus court et que tous ses éléments sont égaux à ceux du tuple plus long, le tuple le plus court est considéré comme inférieur.

```
(1, 2) < (1, 2, 3)  # Renvoie True car le premier tuple est plus court
(1, 3) > (1, 2)     # Renvoie True car 3 est supérieur à 2
(1, 2) == (1, 2)    # Renvoie True car tous les éléments sont égaux
```

5. Parcours à l'aide d'une boucle for

```
>>> t = (1, 2, 3, 4, 5, 6, 7, 8)
>>> for el in t:
...     print(el)
...
1
2
3
4
5
6
7
8
```

6. Les fonctions

Les fonctions comme `max()`, `min()`, `sum()` sont utilisables sur les tuples. Cependant, les tuples étants immuables, il n'est pas possible d'utiliser les fonctions comme `append()`, `insert()`, `remove()`, `reverse()` et `sort()`.

IV. Application des tuples

1. Itérations sur plusieurs valeurs à la fois

La fonction `enumerate()` en Python est un moyen pratique de parcourir une séquence tout en ayant accès à l'index de chaque élément. Elle transforme une séquence, comme une liste ou un tuple, en un objet itérable qui renvoie une paire composée de l'index et de la valeur de l'élément.

```
>>> for valeur in enumerate(nombres):
...     print(valeur, type(valeur))
...
(0, 1) <class 'tuple'>
(1, 2) <class 'tuple'>
(2, 3) <class 'tuple'>
(3, 4) <class 'tuple'>
(4, 5) <class 'tuple'>
>>> for index, valeur in enumerate(nombres):
...     print(f'{index} -> {valeur}')
...
0 -> 1
1 -> 2
2 -> 3
3 -> 4
4 -> 5
```

2. Retour de valeurs multiples

Grace à la notion de tuple, les fonctions peuvent retourner plusieurs valeurs. Les valeurs à retourner sont alors empaqueter dans un tuple qu'on pourra désempaqueter plus tard grâce à l'affectation multiple.

```
>>> def f(a, b, c, d):
...     return 2 * a, 2 * b, 2 * c, 2 * d
>>> n1, n2, n3, n4 = f(1, 2, 3, 4)
>>> print(n1, n2, n3, n4)
2 4 6 8
>>> n1, _, _, n4 = f(1, 2, 3, 4)
>>> print(n1, n4)
2 8
```

Dans le cas où une fonction renvoie plusieurs valeurs et que nous ne sommes intéressés que par certaines de ces valeurs, nous pouvons mettre des underscore à leur position.

```
1. >>> def f(x, y, z):
2. ...     return 2 * x, 2 * y, 2 * z
3. ...
4. >>> print(f(1, 2, 3))
5. (2, 4, 6)
6. >>> a, b, c = f(1, 2, 3)
7. >>> print(a, b, c)
8. 2 4 6
```

Chapitre 5 : Les dictionaries

I. Définition

Un dictionnaire est une collection de paires clé-valeurs qui est ordonnée, modifiable qui n'autorise pas de doublons de clés. Il est possible d'accéder aux différents éléments grâce à leur clé, laquelle pourra être alphabétique, numérique, ou même d'un type composite sous certaines conditions.

II. Création d'un dictionnaire

1. Création d'un dictionnaire vide

Syntaxe :

```
dico = {}
```

2. Création d'un dictionnaire à partir d'une liste de valeurs

Syntaxe :

```
1. >>> dico = {'nom': 'ALITO', 'prenom': 'Jean', 'age': 18}
2. >>> dico
3. {'nom': 'ALITO', 'prenom': 'Jean', 'age': 18}
```

NB :

- Les éléments du dictionnaire sont séparés par des virgules (le tout étant enfermé entre deux accolades ;
- Chacun de ces éléments est constitué d'une paire d'objets : un index et une valeur ;
- On peut utiliser d'autres types d'objets (entiers, floats, et même des tuples) comme des clés ;
- Dans un dictionnaire, les index s'appellent des clés, et les éléments peuvent donc s'appeler des paires clé-valeur.

3. Accès aux éléments du dictionnaire

Syntaxe :

```
nom_dictionnaire[clé]
```

III. Opérations sur les dictionnaires

1. Ajout et modification des éléments

Les dictionnaires étant mutables, nous pouvons ajouter de nouveaux éléments ou modifier des éléments existants à l'aide d'un opérateur d'affectation.

Si la clé est déjà présente, la valeur est mise à jour, sinon une nouvelle paire clé : valeur est ajoutée au dictionnaire.

```
1. >>> personne = {'nom': 'ALITO', 'prenom': 'Jean', 'age': 18}
2. >>> personne['prenom'] = 'Roger'
3. >>> personne
4. {'nom': 'ALITO', 'prenom': 'Roger', 'age': 18}
5. >>> personne['ville'] = 'Lomé'
6. >>> personne
7. {'nom': 'ALITO', 'prenom': 'Roger', 'age': 18, 'ville': 'Lomé'}
```

2. Suppression d'éléments

Le mot-clé `del` pour supprimer des éléments individuels ou le dictionnaire entier lui-même.

```
 1. >>> personne = {'nom': 'ALITO', 'prenom': 'Roger', 'age': 18, 'ville':
'Lomé'}
 2. >>> del personne['ville']
 3. >>> personne
 4. {'nom': 'ALITO', 'prenom': 'Roger', 'age': 18}
 5. >>> del personne
 6. >>> personne
 7. Traceback (most recent call last):
 8.   File "<stdin>", line 1, in <module>
 9. NameError: name 'personne' is not defined
```

3. Itération sur les clés pour obtenir les valeurs

Par défaut, l'itération sur un dictionnaire se fait sur les clés. Grâce aux clés, on peut accéder aux valeurs.

```
1. >>> personne = {'nom': 'ALITO', 'prenom': 'Roger', 'age': 18, 'ville': 'Lomé'}
2. >>> for key in personne:
3. ...     print(f'{key}: {personne[key]}')
4. ...
5. nom: ALITO
6. prenom: Roger
7. age: 18
8. ville: Lomé
```

4. Différence entre deux dictionnaires

La différence entre deux dictionnaires peut être calculée grâce à l'opérateur différence des sets. Pour ce faire, il faut convertir les deux dictionnaires en set puis appliquer l'opérateur différence. Il devient ainsi possible d'utiliser tous les autres opérateurs vus sur les sets dans le cadre de ce cours.

```
1. >>> personne1 = {'nom': 'Nom 1', 'prenom': 'Prenom 1', 'age': 20}
2. >>> personne2 = {'nom': 'Nom 2', 'prenom': 'Prenom 2'}
3. >>> set(personne1) - set(personne2)
4. {'age'}
```

IV. Méthodes de dictionnaire

Un certain nombre de méthodes sont disponibles et utilisables sur les objets de type dictionnaires.

1. Les méthodes keys, values et items

a. La méthode keys

La méthode `keys()` renvoie la liste des clés utilisées dans le dictionnaire :

```
1. >>> personne = {'nom': 'ALITO', 'prenom': 'Roger', 'age': 18}
2. >>> print(personne.keys())
3. dict_keys(['nom', 'prenom', 'age'])
4. >>> print(list(personne.keys()))
5. ['nom', 'prenom', 'age']
```

b. La méthode values

La méthode `values()` renvoie la liste des valeurs mémorisées dans le dictionnaire :

```
1. >>> print(personne.values())
2. dict_values(['ALITO', 'Roger', 18])
3. >>> print(list(personne.values()))
4. ['ALITO', 'Roger', 18]
```

c. La méthode items

La méthode `items()` extrait du dictionnaire une liste équivalente de tuples :

```
>>> print(personne.items())
dict_items([('nom', 'ALITO'), ('prenom', 'Roger'), ('age', 18)])
>>> for cle, valeur in personne.items():
...     print(f"Clé : {cle}, valeur: {valeur}")
...
Clé : nom, valeur: ALITO
Clé : prenom, valeur: Roger
Clé : age, valeur: 18
```

2. La méthode get

La méthode `get()` dans un dictionnaire Python est utilisée pour accéder à la valeur associée à une clé donnée. Elle a deux avantages principaux par rapport à l'accès direct aux éléments par les crochets `[]` :

Gestion des clés manquantes : Si la clé n'existe pas dans le dictionnaire, `get()` retourne une valeur par défaut, qui est `None` par défaut, au lieu de lever une `KeyError`.

Il est possible également de spécifier une valeur par défaut différente.

```
>>> personne = {'nom': 'ALITO', 'prenom': 'Roger', 'age': 18}
>>> personne.get('age')
18
>>> personne.get('ville', 'Lomé')
'Lomé'
```

3. La méthode update

Elle permet de mettre à jour le dictionnaire avec les paires clé/valeur de dict en remplaçant les clés existantes.

```
>>> personne = {'nom': 'ALITO', 'prenom': 'Roger', 'age': 18}
>>> personne.update({'age': 20, 'ville': 'Lomé'})
>>> personne
{'nom': 'ALITO', 'prenom': 'Roger', 'age': 20, 'ville': 'Lomé'}
```

4. La méthode popitem

La méthode `popitem()` dans un dictionnaire Python est utilisée pour retirer et retourner une paire (clé, valeur). Depuis **Python 3.7**, cette méthode retire et retourne la dernière paire clé-valeur insérée dans le dictionnaire. Dans les versions antérieures à **3.7**, elle retirait et retournait une paire clé-valeur de manière aléatoire.

Elle lance une exception de type **KeyError** si le dictionnaire est vide.

```
1. >>> personne = {'nom': 'ALITO', 'prenom': 'Roger', 'age': 18}
2. >>> personne.popitem()
3. ('age', 18)
```

5. La méthode fromkeys

La méthode `fromkeys()` est une méthode de classe dans les dictionnaires Python qui est utilisée pour créer un nouveau dictionnaire à partir d'une séquence d'éléments avec une valeur fournie par l'utilisateur. Cette méthode accepte deux paramètres, dont l'un est obligatoire et l'autre est optionnel. Elle retourne le nouveau dictionnaire où les éléments de la séquence sont considérés comme les clés du dictionnaire.

```
1. >>> cles = ['nom', 'prenom', 'age']
2. >>> personne = dict.fromkeys(cles)
3. >>> personne
4. {'nom': None, 'prenom': None, 'age': None}
5. >>> cles = ['prix ht', 'prix ttc', 'tva']
6. >>> achat = dict.fromkeys(cles, 0.0)
7. >>> achat
8. {'prix ht': 0.0, 'prix ttc': 0.0, 'tva': 0.0}
```

V. Liste de dictionnaires

En créant une liste de dictionnaires qui possèdent les mêmes clés, on obtient une structure qui ressemble à une base de données.

```
1. >>> personne1 = {'nom': 'Nom 1', 'prenom': 'Prenom 1', 'age': 18}
2. >>> personne2 = {'nom': 'Nom 2', 'prenom': 'Prenom 2', 'age': 20}
3. >>> personne3 = {'nom': 'Nom 3', 'prenom': 'Prenom 3', 'age': 30}
4. >>> base = [personne1, personne2, personne3]
5. >>> base
6. [{'nom': 'Nom 2', 'prenom': 'Prenom 2', 'age': 20}, {'nom': 'Nom 2', 'prenom':
'Prenom 2', 'age': 20}, {'nom': 'Nom 3', 'prenom': 'Prenom 3', 'age': 30}]
```

VI. Les tris sur les dictionnaires

1. Tri par clés

La fonction `sorted()` peut être utilisé pour les clés d'un dictionnaire.

```
1. >>> nombres = {'b': 2, 'c': 3, 'a': 1}
2. >>> sorted(nombres)
3. ['a', 'b', 'c']
```

2. Tri par valeurs

La fonction `sorted()` en combinaison avec une fonction `lambda` permet de trier un dictionnaire par ses valeurs. Une fois le tri effectué, ce sont les clés qui sont retournées et non les valeurs du dictionnaire.

a. Grâce à la méthode get

```
1. >>> nombres =  {'a': 3, 'b': 1, 'c': 2}
2. >>> sorted(nombres, key=nombres.get)
3. ['b', 'c', 'a']
```

b. Grâce à une fonction lambda

```
1. >>> nombres =  {'a': 3, 'b': 1, 'c': 2}
2. >>> sorted(nombres.items(), key=lambda el: el[1])
3. [('b', 1), ('c', 2), ('a', 3)]
```

Chapitre 6 : Résumé des structures de données de type container

Critères\Types	Chaîne de caractères	Range	Listes	Sets	Frozensets	Tuples	Dictionnaires
Définition	`s = ""`	`range(10)`	`[1, 2]`	`{1, 2}`	`frozenset({1, 2})`	`(1, 2)`	`{"a": 1, "b": 2}`
Test d'appartenance et fonction len()	✅	✅	✅	✅	✅	✅	✅
Itérable	✅	✅	✅	✅	✅	✅	✅ (sur les clés)
Ordonné	✅	✅	✅	❌	❌	✅	✅ (Python 3.7)
Indexable	✅	✅	✅	❌	❌	✅	❌
Possibilité d'insérer ou supprimer des éléments	❌	❌	✅	✅	❌	❌	✅
Possibilité de modifier des éléments	❌	❌	✅	❌	❌	❌	✅
Possibilité de stocker des valeurs dupliquées	✅	❌	✅	❌	❌	✅	✅

I. Quelques fonctions supplémentaires sur les chaines

Nom de la fonction	Fonctionnement
`find("sous-chaine", debut, fin)`	Trouve la position de sous-chaine dans une chaîne
`islower()`	Renvoie **`True`** si toutes les lettres de la chaîne sont en minuscule, sinon **`False`**
`isupper()`	Retourne **`True`** si toutes les lettres de la chaîne sont en majuscules, sinon **`Faux`**.
`lower()`	Renvoie la nouvelle chaîne avec toutes les lettres converties en minuscules.
`upper()`	Renvoie la nouvelle chaîne avec toutes les lettres converties en majuscules.
`swapcase()`	Inverse la casse de la chaîne, c'est-à-dire que les majuscules sont converties en minuscules et vice versa.
`title()`	Convertit la chaîne en casse du titre, c'est-à-dire que la première lettre de chaque mot de chaîne est en majuscule et que tous les autres sont en minuscule.

`strip()`	Supprime tous les caractères de début et de fin mentionnés dans son argument (La valeur par défaut est l'espace).
`lstrip()`	Supprimer tous les caractères principaux mentionnés dans son argument se trouvant au début de la chaine (La valeur par défaut est l'espace).
`rstrip()`	Même chose que **`lstrip()`** mais à droite.
`min("chaîne")`	Renvoie le caractère minimum dans l'alphabet à partir de la chaîne.
`max("chaîne")`	Renvoie le caractère maximum dans l'alphabet à partir de la chaîne.
`replace ("sous-chaine","new-chaine",max)`	Permet de remplacer la sous-chaîne par une nouvelle sous-chaîne "new-chaine" dans la chaîne. Cette fonction a 3 arguments. La chaîne à remplacer, nouvelle chaîne qui remplacerait et valeur max indiquant le nombre de remplacements à effectuer (par défaut, illimité).

II. Quelques méthodes disponibles avec l'objet file

Méthode	Description
`close()`	Fermer un fichier ouvert. Cela n'a aucun effet si le fichier est déjà fermé.
`fileno()`	Renvoie un nombre entier (descripteur) du fichier.
`flush()`	Vider le tampon d'écriture du flux de fichiers.
`isatty()`	Renvoie **`True`** si le flux de fichiers est interactif.
`read(n)`	Lire au plus **`n`** caractères du fichier. Lit jusqu'à la fin du fichier s'il est négatif ou nul.
`readable()`	Renvoie **`True`** si le flux de fichiers peut être lu.
`readline(n=-1)`	Lire et retourner une ligne du fichier. Lit au plus **`n`** octets si spécifié.
`readlines(n=-1)`	Lire et retourner une liste de lignes du fichier. Lit au plus **`n`** octets / caractères si spécifié.
`seek(offset,from=SEEK_SET)`	Modifiez la position du fichier en octets décalés (**`offset`**), en référence à "from" (**`début`**, **`actuel`**, **`fin`**).
`seekable()`	Renvoie **`True`** si le flux de fichiers prend en charge l'accès aléatoire.
`tell()`	Renvoie la position actuelle du curseur dans un fichier ouvert
`truncate(size=None)`	Elle est utilisée pour tronquer, ou réduire, la taille d'un fichier ouvert à la position actuelle du curseur. Si la taille (**`size`**) n'est pas spécifiée, redimensionner à l'emplacement actuel du cureur.
`writable()`	Renvoie **`True`** si le flux de fichier peut être écrit.
`write(s)`	Ecrit la chaîne "s" dans le fichier et renvoie le nombre de caractères écrits.
`writelines(lines)`	Ecrire une liste de lignes(lines) dans le fichier.
`closed`	Renvoie **`True`** si le fichier est fermé, **`False`** dans le cas contraire.